

Classification, Challenges, and Automated Approaches to Handle Non-Functional Requirements in ML-Enabled Systems: A Systematic Literature Review

VINCENZO DE MARTINO, Software Engineering (SeSa) Lab, University of Salerno, Italy

FABIO PALOMBA, Software Engineering (SeSa) Lab, University of Salerno, Italy

Although machine learning (ML) is so pervasive and diffused that virtually no application can avoid its use, its enormous potential is constantly threatened by non-functional requirements, such as sustainability. In particular, we noticed the lack of a comprehensive synthesis of the research efforts done so far and how these may drive further research. In this paper, we propose a systematic literature review targeting three key aspects such as (1) the classification of the non-functional requirements investigated so far, (2) the challenges to face when dealing with them, and (3) the automated approaches proposed in literature to support practitioners when optimizing them in practice. Through the combination of well-established guidelines for conducting systematic literature reviews and additional search criteria, we survey a total amount of 50 research articles. Our findings report that current research identified 29 different non-functional requirements, which can be grouped into six main classes. We also deliver a catalog of over 20 software engineering challenges that further research should consider, besides an overview of the automated approaches researchers proposed to support practitioners when optimizing non-functional requirements of machine learning-enabled systems. We conclude our work by distilling implications and a future outlook on the topic.

CCS Concepts: • **Computing methodologies** → **Machine learning**; • **Software and its engineering** → **Extra-functional properties**.

Additional Key Words and Phrases: Software Engineering for Artificial Intelligence, Non-Functional Requirements, Systematic Literature Reviews.

ACM Reference Format:

Vincenzo De Martino and Fabio Palomba. 2023. Classification, Challenges, and Automated Approaches to Handle Non-Functional Requirements in ML-Enabled Systems: A Systematic Literature Review. 1, 1 (May 2023), 44 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Machine learning (ML) is now, more than ever, no longer restricted by computational limits, being used in theory, experiment, and simulation [68, 71]. On the one hand, companies and individuals increasingly rely on the outcome of machine learning models to make informed decisions [100] or automate tasks that would take substantial human workload [70]. On the other hand, machine learning-intensive systems, i.e., systems that embed machine learning solutions, have been recently deployed in multiple domains, with some recent applications showing highly efficient and accurate performance [60, 67]. As such, the pervasiveness of machine learning-intensive systems is expected to increase further in the coming years in multiple domains [69, 98].

Authors' addresses: Vincenzo De Martino, vdemartino@unisa.it, Software Engineering (SeSa) Lab, University of Salerno, Italy; Fabio Palomba, fpalomba@unisa.it, Software Engineering (SeSa) Lab, University of Salerno, Italy.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2023 Association for Computing Machinery.

XXXX-XXXX/2023/5-ART \$15.00

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Nonetheless, such a pervasiveness is constantly threatened by multiple concerns, which are often not related to the overall accuracy, but rather to non-functional attributes affecting the level of reliability, trustworthiness, and sustainability of machine learning-intensive systems [29, 33, 53]. It is therefore not surprising that the software engineering research community—and specifically the software engineering for artificial intelligence (SE4AI) research branch—has been investing notable efforts in understanding non-functional requirements of machine learning-intensive systems, other than proposing methods and instruments to support practitioners when dealing with them [12, 28, 37]. This effort is also stimulated by government and funding agencies, which are more and more willing to invest in the matter, e.g., the European Union has recently approved the so-called *Artificial Intelligence Act*,¹ which aims at promoting research on themes connected to the improvement of non-functional attributes of artificial intelligence-based software systems.

Recent advances in the field of SE4AI contributed to the development of a consistent body of knowledge with respect to the management of multiple non-functional requirements, including fairness [16, 23, 96], security [30, 51], privacy [49, 78], and more [43]. While recognizing the relevant advances made over the last years, our research identifies a notable key limitation.

▲ *Despite the current, extensive body of knowledge produced by the SE4AI research community with respect to the management of non-functional requirements of ML-intensive systems, there is still a lack of a comprehensive, systematic synthesis of the current knowledge on the non-functional attributes affecting machine learning-enabled systems, the challenges faced when dealing with them, and the automated approaches proposed to support practitioners when optimizing them throughout the software development lifecycle.*

An improved understanding of these aspects may have critical implications for researchers and practitioners. First, researchers might learn more about the current state of the art, possibly identifying neglected research angles that would be worth further investigating. At the same time, practitioners may have a comprehensive overview of the instruments that researchers have been providing to support the analysis and optimization of non-functional requirements, possibly accelerating the technological transfer of academic prototypes to industry.

In this paper, we conduct a systematic literature review (SLR) on non-functional requirements of machine learning-intensive software systems. Our work follows well-established guidelines [39, 88] and additional search criteria based on seed set identification [63] to comprehensively synthesize existing research. From an initial set composed of over 2,500 hits, and after applying multiple snowballing rounds and additional data collection procedures, we ended up analyzing more than 20,577 research results. Through the application of exclusion/inclusion criteria and a rigorous quality assessment, we finally selected 50 papers. The key findings of our literature review report the classification of 29 non-functional requirements, grouped in six classes. In addition, we provide a novel catalog composed of more than 20 software engineering challenges to deal with them, other than an overview of the automated approaches proposed by researchers to support practitioners when optimizing non-functional requirements throughout the software lifecycle. We conclude the paper by elaborating on the implications of our results, along with the actionable items that readers of our work may (re-)use to analyze further the problem of non-functional requirements in machine learning-intensive software systems.

Structure of the paper. Section 2 analyzes the related work. Section 3 describes the research goals and the methods applied to address them, while Section 4 discusses the results achieved. The

¹The European Union Artificial Intelligence Act: <https://artificialintelligenceact.eu>.

Table 1. Comparison with the Closest Related Work.

Related Work	Main Focus	Commonalities and differences
Martinez-Fernandez et al. [57]	A systematic mapping review targeting all previous research on software engineering for artificial intelligence. The authors surveyed 248 studies, classifying the available research according to the SWEBOK areas [14].	• Similar research approach to the search, yet with some differences in terms of seed set identification; • A broader analysis of current research, with no specific focus on non-functional requirements; • While the work includes part of the research papers we analyze, it does not provide insights into the challenges, practices, and methods to deal with non-functional requirements of machine learning-intensive systems.
Habibullah et al. [28]	A non-systematic analysis of non-functional requirements of machine learning-intensive systems. The authors focused on (i) the identification of clusters of non-functional requirements, (ii) the estimation of the amount of relevant studies for a subset of non-functional requirements, and (iii) the definition of the scope of non-functional requirements.	• We approach the literature search with a systematic approach, hence making a more comprehensive analysis; • We did not limit ourselves to the estimation of the current interest in the matter but provided a systematic investigation; • We let additional factors arise, including challenges, practices, and methods to deal with non-functional requirements of machine learning-intensive systems.
Horkoff [33]	An experience report on the challenges that the Requirements Engineering (RE) research community is called to face when addressing non-functional attributes of machine learning-intensive systems.	• We approach the literature search with a systematic approach, hence making a more comprehensive analysis; • We let additional factors arise, including a comprehensive set of non-functional requirements for machine learning-intensive systems, other than practices and methods to deal with them; • We let the challenges to deal with non-functional requirements emerge from the scientific literature on the matter, hence approaching the research differently.
Habibullah and Horkoff [29]	An industrial, interview-based study aiming at assessing (i) the identification and measurement of non-functional requirements, (ii) the importance of non-functional requirements in the industry, and (iii) the challenges associated with the identified non-functional requirements.	• We let our findings emerge from the scientific literature on the matter, hence approaching the research differently; • We let additional factors arise, including a comprehensive set of non-functional requirements for machine learning-intensive systems, other than practices and methods to deal with them; • We did not limit ourselves to the observations made by the industry practitioners but provided a systematic investigation of the current relevance of non-functional requirements.
Ahmad et al. [5]	A systematic mapping study on the requirements specification and modeling approaches and tools.	• The work targets requirements engineering from the perspective of specification and modeling, while ours focuses on non-functional attributes; • The research method is similar, yet we perform a systematic literature review rather than a mapping study, hence being more restrictive in terms of quality assessment of the primary studies;

implications and actionable insights are elaborated in Section 5, with the limitations reported in Section 6. Finally, Section 7 concludes the paper and outlines our future research agenda.

2 RELATED WORK AND MOTIVATION

To the best of our knowledge, no systematic literature review has been conducted with the aim of classifying non-functional requirements of machine learning-enabled systems and summarizing the challenges and automated approaches to deal with them. At the same time, it is important to point out that some secondary studies recently attempted to (1) synthesize the research on software engineering for artificial intelligence [57], (2) explore, in a preliminary fashion, the relevance and research interest around non-functional requirements of machine learning-intensive systems [28], and (3) summarize some of the key academic and industrial challenges when dealing with non-functional requirements in industry [29, 33]. These works are clearly the most closely related

to ours. This section describes the major differences and limitations of current literature that motivated our research. A summary is provided in Table 1.

First and foremost, Martinez-Fernandez et al. [57] conducted a systematic mapping study of the research on software engineering for artificial intelligence. The main goal of the work was to provide a comprehensive schema representing the elements composing the field of SE4AI, from requirements engineering to verification and validation. In other terms, the systematic mapping study had a pretty broad objective and aimed at covering all the research on the matter. As such, there are multiple differences for our study. While our scope is limited to non-functional requirements, we aim to address the matter in a more detailed fashion by letting emerge a complete set of non-functional requirements discussed in the literature, other than the challenges and automated approaches proposed to deal with them. Secondly, ours is a systematic literature review rather than a mapping study: as such, there are intrinsic, methodical differences in the search process conducted and in the criteria used to select the relevant pieces of research. Third, we additionally tuned the search process to produce an extensive set of seed papers, as further discussed in Section 3—hence attempting to strengthen the completeness of the search process.

Habibullah et al. [28] recently investigated the topic of non-functional requirements of machine learning-enabled systems under three perspectives such as (1) the clustering of non-functional attributes based on shared characteristics; (2) the estimation of the number of relevant studies that investigated aspects connected to non-functional attributes; and (3) the definition of the scope of non-functional requirements. Habibullah et al. [28] share the same overall objective of our paper, i.e., an improved understanding of non-functional requirements of machine learning-intensive systems. Nonetheless, we aimed to conduct a comprehensive, systematic literature review. At the same time, we focused on a larger set of research angles, including an investigation of the challenges and approaches proposed to deal with non-functional requirements.

Horkoff [33] discussed the challenges that the requirements engineering research community would be called to face to address themes connected to non-functional requirements of machine learning-intensive systems. The author identified several challenges based on the extensive experience accumulated in the industry over the years. With respect to this paper, our systematic literature review aims to collect comprehensive pieces of information coming from the scientific community to provide insights into how researchers have addressed the problem so far. At the same time, it is worth saying that we could corroborate the relevance of some of the challenges identified by Horkoff [33] through the evidence coming from our work.

Habibullah and Horkoff [29] conducted an interview-based study with ten machine learning engineers to elicit the practices used to face non-functional requirements. In particular, the authors were interested in collecting information about the identification and measurement mechanisms put in place, the importance of various non-functional requirements from their perspective, and the challenges associated with the identified non-functional requirements. This work can be therefore seen as complementary to our systematic literature review. On the one hand, we enlarge the knowledge of non-functional requirements in machine learning-enabled systems by synthesizing the current literature from various perspectives. On the other hand, we could corroborate and further extend the findings on the challenges identified through a different research method.

Finally, Ahmad et al. [5] recently performed a systematic mapping of the research on requirements engineering for artificial intelligence. The authors surveyed the approaches to specify requirements and the frameworks/tools/methods to model them. Our work is clearly complementary, as we focus on non-functional attributes, attempting to classify them and synthesize the current knowledge on the challenges and approaches to deal with them.

While the papers discussed above are closely connected to the work proposed herein, it is also worth mentioning the existence of an ever-increasing number of secondary studies targeting multiple aspects of software engineering for artificial intelligence research.

A consistent amount of systematic literature reviews focused on the synergies between the artificial intelligence and software engineering research communities [85], other than on the software engineering challenges and solutions for developing artificial intelligence systems [26, 43, 53, 64]. With respect to them, our work focuses on non-functional requirements, hence providing finer-grained pieces of information and insights on the next steps that researchers should consider to better support practitioners.

Other researchers targeted the quality assurance problem, which is deemed one of the most relevant and complex for the SE4AI research community. In particular, we identified systematic literature review and empirical investigations into the field of software quality [25, 59], design patterns [75, 87], software architecture [76], and testing [13, 15, 72, 97]. Finally, recent systematic literature reviews have been conducted with the aim of understanding the deployment strategies for artificial intelligence systems [36], and model-based development approaches [52].

The substantial efforts put by researchers in conducting systematic investigations on software engineering for artificial intelligence further motivate our work, as they let arise the collective, emerging interest around these matters. Our systematic literature review aims, therefore, to provide additional insights to the research community by studying the non-functional requirements of machine learning-enabled systems that have been studied over the last years.

3 RESEARCH METHOD

The ultimate *goal* of the study was to systematically survey the current research on non-functional requirements of machine learning-enabled systems, with the *purpose* of providing researchers in the field of SE4AI with actionable items and insights that they can exploit to further explore the matter and improve the support provided to machine learning engineers. Specifically, the research was conducted in terms of (1) the classification of the non-functional requirements of ML-enabled systems, (2) the challenges that non-functional requirements pose during the development of ML-enabled systems, and (3) the automated approaches that may support practitioners throughout the development lifecycle. The *perspective* is of fresh Ph.D. students, senior researchers, and practitioners. The former are interested in mapping the research literature that focuses on addressing non-functional requirements in machine learning-enabled systems so that they can possibly identify neglected research areas to further investigate. Senior researchers are interested in mapping the state of the art in order to identify opportunities for research proposals and industrial collaborations. Finally, practitioners are interested in understanding the challenges to face when optimizing non-functional requirements during the development of ML-enabled systems, other than the current support provided by researchers, hence possibly identifying chances for technological transfer of those approaches in the industry.

To address our goal, we designed and executed a systematic literature review (SLR), which is a process through which the existing scientific papers on a subject of interest are rigorously identified, selected, and analyzed to address one or more research questions [39]. To make our SLR as complete as possible, we first followed the well-established guidelines proposed by Kitchenham et al. [38].

However, as further elaborated in the remainder of this section, the comprehensive search of the articles connected to non-functional requirements of machine learning-intensive systems could not be limited to the papers identified through the guidelines by Kitchenham et al. [38]. We were indeed required to extend the scope of our search process because the search terms which were reasonable to use to look for relevant articles were found not to be enough. As a consequence, we boosted the search process by means of two additional steps. On the one hand, we systematically

screened the research articles published in top-tier software engineering venues in an effort to identify a set of seed papers to further process, i.e., the identification of seed papers has been shown to be an effective alternative when investigating research angles for which the standard guidelines do not enable a comprehensive search [90]. On the other hand, we provided additional rigor to the analysis by integrating the so-called snowballing procedure [89], i.e., the iterative scanning of the incoming and outcoming references of the primary studies done to identify additional relevant sources of information. Therefore, our systematic literature review can be considered “hybrid” [63], namely a systematic analysis that combines traditional search strategies with additional search steps and snowballing. In terms of reporting, we followed the *ACM/SIGSOFT Empirical Standards*² and, in particular, the “General Standard” and “Systematic Reviews” guidelines.

3.1 Research Objectives and Questions

The Goal-Question-Metric (GQM) approach [17] was initially employed to purposefully measure the goals to be achieved and relate them to the data analyzed. More specifically, the overall research objectives established by this systematic literature review were the following:

- ⊙ **Objective 1.** *Providing a systematic classification of the non-functional requirements of ML-enabled software defined in the literature.*
- ⊙ **Objective 2.** *Providing a systematic classification of the challenges to face when dealing with non-functional requirements of ML-enabled systems.*
- ⊙ **Objective 3.** *Synthesizing the automated approaches proposed to handle non-functional requirements in ML-enabled software throughout the development lifecycle.*

Starting from these objectives, we defined more specific research questions (**RQs**) that could drive our search process and analysis. These are presented in Table 2.

Table 2. Research Questions of the Systematic Literature Review.

Research Question	Description
RQ₁	What are the non-functional requirements of ML-intensive software considered by researchers?
RQ₂	What are the challenges of dealing with non-functional requirements of ML-intensive software?
RQ₃	What automated approaches are proposed to deal with non-functional requirements of ML-intensive systems?

More particularly, **RQ₁** addressed the first objective of the study and aimed at systematically classifying the non-functional requirements that researchers have defined. By addressing this research angle, we aimed to contribute to previous research on the classification of non-functional requirements of ML-intensive systems [28]. Indeed, while previous efforts in this respect have been conducted, there is still not a systematic classification - this represents a key contribution to our work. The second objective was mapped onto **RQ₂**. This research question elicited the

²The *ACM/SIGSOFT Empirical Standards*: <https://github.com/acmsigsoft/EmpiricalStandards>.

major challenges that non-functional requirements pose during the development of ML-enabled systems. On the one hand, this research question allowed us to understand the rationale behind the existence of the automated approaches proposed in the literature. On the other hand, **RQ₂** was able to enlarge the body of knowledge on the challenges to face when dealing with non-functional requirements, possibly informing researchers and practitioners on the aspects to further consider during the development of these systems. Finally, the third objective was mapped onto **RQ₃**. The final step of our systematic literature review aimed at classifying which are the existing automated approaches, in an effort to provide an overview of the current status of the research on the matter. First, the results of this research question may inform researchers about neglected research areas to investigate. Second, **RQ₃** may be useful to practitioners to overview the currently available support, possibly finding technological transfer opportunities.

3.2 Research Method to Conduct the Systematic Literature Search

As a first step to address the goals of our study, we applied the guidelines by Kitchenham et al. [39] to identify primary studies targeting non-functional requirements of ML-enabled systems.

3.2.1 Research Query definition. To develop an effective search string, we first extracted relevant terms from the research questions, identifying the keywords from the **RQs** [39]. For all the terms, we then elaborated on the alternative spellings and synonyms. Afterward, we used boolean operators to assemble the search string, i.e., we used the ‘OR’ operator for the concatenation of alternative spellings and synonyms, while the ‘AND’ operator for the concatenation of relevant terms. More specifically, we elaborated on the following search string:

Search String.

((“Machine Learning”) OR (“Artificial Intelligence”) OR (“Deep Learning”) OR (“ml”) OR (“ai”) OR (“dl”)) AND ((“nfr”) OR (“Non-Functional Requirement*”) OR (“Non Functional Requirement*”))

There are some considerations to make on the definition of the search string. In the first place, it included terms connected to both machine and deep learning but also to artificial intelligence. This was done to deal with the lack of a standard terminology: it is indeed possible that a consistent amount of researchers used more generic terms, like “Artificial Intelligence”, to indicate the analysis of ML-enabled systems. In the second place, it is worth discussing that we did not include terms connected to the development of automated approaches. We made this choice on purpose, as the first two research questions were not strictly connected to the automated approaches proposed in the literature but rather aimed at providing an overview of the non-functional requirements defined and the challenges they pose to the development of ML-enabled systems. In other terms, we preferred to have a more generic search string in an effort to increase the recall of the search process, being able to collect a larger amount of papers. Of course, this decision impacted the precision of the search process and, subsequently, the effort required to apply the exclusion and inclusion criteria. Nonetheless, we accepted this compromise to make sure to include all relevant sources in our investigation.

3.2.2 Search Databases. As a second step of our investigation, we selected the databases to collect relevant resources. We applied the search on three well-known databases such as *ACM Digital Library*,³ *Scopus*,⁴ and *IEEEExplore*.⁵ These digital libraries are often used to carry out systematic

³Link to *ACM Digital Library*: <https://dl.acm.org>.

⁴Link to *Scopus*: www.scopus.com.

⁵Link to *IEEEExplore*: <http://ieeexplore.ieee.org>.

literature and mapping studies, as they provide a comprehensive set of resources to conduct them. It is worth noticing that *Scopus* indexes all the papers published by relevant publishers such as *Springer* and *Elsevier* - this is the reason why we did not include the *SpringerLink*⁶ and *ScienceDirect*⁷ databases. At the same time, we still opted for the inclusion of *ACM Digital Library* and *IEEEExplore*. This was done on the basis of the fact that the proceedings of some relevant ACM and IEEE conferences might not have been indexed by *Scopus* and, for this reason, we might have missed relevant resources for our study.

3.2.3 Exclusion and Inclusion criteria. To be considered useful for the goals of the systematic literature review, the papers retrieved from the search process had to be assessed against the following exclusion and inclusion criteria [38].

Exclusion criteria. The resources that met the constraints reported below were excluded:

- Papers not written in English;
- Duplicated papers;
- Papers whose full-text read was not available;
- Paper not published or not peer-reviewed;
- Workshop, systematic, survey papers;
- Short papers, with a page count of less than five pages;
- Papers that did not fall into themes of computer science and computer engineering;
- Papers published before 2012;
- Conference papers which were later extended as journal submissions;
- Papers out of scope;

Inclusion criteria. The resources that met the constraints reported below were included:

- Papers defining types of non-functional requirements of ML-enabled systems;
- Papers presenting approaches or tools to analyze non-functional requirements of ML-enabled systems;
- Papers presenting the domain in which non-functional requirements for ML-enabled systems are applied;
- Papers presenting methodologies to optimize non-functional requirements problems in ML-enabled systems.

3.3 Research Method to Conduct the Seed Set Search

As further detailed in Section 3.7, the research method implemented through the guidelines by Kitchenham et al. [39] was found to be insufficient in finding relevant articles. Similarly to Habibullah et al. [29], we found out that the relevant papers might have referred to specific non-functional requirements, e.g., “*fairness*”, rather than to the more generic term “*non-functional requirement*”. As such, the traditional systematic literature review approach might not have ensured completeness.

While a possible, reasonable mitigation strategy might have been that of enlarging the scope of the search query to include specific non-functional requirements, we would have faced two issues. First, we would have been bound to the inclusion of the non-functional requirements explicitly classified in previous studies while defining the search string: as a consequence, we could not have satisfactorily addressed **RQ₁**, not being able to classify additional non-functional requirements emerging from primary studies. Perhaps more importantly, the systematic literature review would have become prohibitive in terms of effort. For instance, Habibullah et al. [29] estimated the number of hits for a search query that includes all classified non-functional requirements in over 200,000

⁶Link to *SpringerLink*: <https://link.springer.com/>.

⁷Link to *ScienceDirect*: <https://www.sciencedirect.com>.

articles: on the one hand, such a search query would have had a low precision, identifying a high amount of irrelevant resources; on the other hand, the application of exclusion and inclusion criteria over such a large set of candidate articles would have required an excessive effort. As such, we had to identify an alternative solution.

Table 3. Conferences and journals considered in the scope of the seed set identification process.

Venue	Name	Ranking
Journal	<i>IEEE Transactions on Software Engineering (TSE).</i>	Q1
Journal	<i>ACM Transactions on Software Engineering and Methodology (TOSEM).</i>	Q1
Journal	<i>Empirical Software Engineering (EMSE).</i>	Q1
Journal	<i>Journal of Systems and Software (JSS).</i>	Q1
Journal	<i>Information and Software Technology (IST).</i>	Q1
Conference	<i>IEEE/ACM International Conference on Software Engineering (ICSE).</i>	A*
Conference	<i>Joint European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE).</i>	A*
Conference	<i>IEEE/ACM Automated Software Engineering Conference (ASE).</i>	A*
Conference	<i>IEEE International Conference on Software Maintenance and Evolution (ICSME).</i>	A
Conference	<i>IEEE International Conference on Software Testing, Verification and Validation (ICST).</i>	A
Conference	<i>ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA).</i>	A
Conference	<i>IEEE International Requirements Engineering Conference (RE).</i>	A
Conference	<i>IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER).</i>	A
Conference	<i>IEEE International Conference on Program Comprehension (ICPC).</i>	A
Conference	<i>IEEE International Working Conference on Mining Software Repositories (MSR).</i>	A

Particularly, we approached the problem by means of the so-called *seed set identification*. This is the process through which a researcher systematically screens the research papers published in top conferences and journals over a given time period in an effort to identify additional resources relevant to the research they are conducting. Wohlin et al. [90] recently showed that this approach could be used to effectively complement a systematic literature review in cases where the standard guidelines cannot lead to a comprehensive search. In our context, we proceeded as follows. In the first place, we identified the most relevant software engineering conferences and journals, relying on (1) the CORE Ranking system⁸ to select A* and A conferences, and (2) the Scimago Journal Ranking⁹ to identify the software engineering journals falling into the first quartile of all journals in the ‘Software’ category. As such, we selected the conferences and journals presented in Table 3. In the second place, we scanned all the papers published at those conferences starting from 2022 backward until 2018, namely until reaching saturation—the time was in line with the results reported by Martinez-Fernandez et al. [57], who discovered that the research field of SE4AI has been active since 2018. As a final step, we analyzed each conference website, and for journals, we searched for papers via the DBLP,¹⁰ the most extensive computer science bibliography library. We

⁸The Computing Research and Education Association of Australasia: <https://www.core.edu.au/>

⁹The Scimago Journal Ranking: <https://www.scimagojr.com>.

¹⁰The DBLP: <https://dblp.org/>.

applied the same exclusion and inclusion criteria defined for the search process conducted using the standard guidelines (see Section 3.2.3).

3.4 Research Method to Conduct the Snowballing Process

The third step conducted to ensure the completeness of the search process revolved around the so-called *forward and backward snowballing*. This is the procedure through which a researcher systematically scans the incoming and outcoming references of the primary studies with the aim of identifying new relevant resources to address the research questions [88]. In our case, the primary studies identified as a consequence of the application of the exclusion and inclusion criteria on both the studies retrieved using the standard guidelines and the seed search were scanned. In particular, we applied an iterative procedure in which all incoming and outcoming references of the primary studies were first considered. Afterward, we reiterated the procedure for the newly acquired studies in an effort to identify additional resources. Overall, three rounds of backward and three rounds of forward were conducted: we stopped at three as we reached saturation, namely, we did not identify any additional papers to include. From a practical perspective, the snowballing steps were conducted through the use of *Google Scholar*,¹¹ an academic search engine which simplifies the analysis of incoming and outcoming references of research papers.

3.5 Research Method to Conduct the Quality Assessment

Once we had completed the application of the three complementary search processes described in the previous sections, we conducted a quality assessment of the resources that successfully passed the inclusion criteria. This step is recommended to ensure that the primary studies would have sound and reliable pieces of information to address the research questions of the study, while the papers that are not found to be of sufficient quality are discarded from the analysis [39].

The implementation of the quality assessment process started with the definition of qualitative questions aiming at operationalizing the main pieces of information that a primary study should have had in order to be useful to address our research questions. These were defined according to the research objectives initially defined:

- **Q1:** *Is the non-functional requirement well defined?*
- **Q2:** *If the primary study defined a challenge to deal with a non-functional requirement, is this well defined?*
- **Q3:** *If the primary study proposes an automated approach to deal with a non-functional requirement, are the data preparation steps well-defined?*
- **Q4:** *If the primary study proposes an automated technique to deal with a non-functional requirement, is the pipeline used to optimize it described?*

In particular, **Q1** was the instrument to ensure a proper answer to **RQ₁**, while **Q2** mapped our requirements to address **RQ₂**. Finally, **Q3** and **Q4** were relevant to understand the way automated approaches were fed and engineered and, as a consequence, to address **RQ₃**.

When assessing the primary studies against each of the qualitative questions, previous systematic literature reviews (e.g., [5, 8, 20]) assigned a boolean value to indicate whether a study had or not the quality required with respect to a property considered. However, assessing the primary studies through boolean values might be challenging, other than possibly threatening the validity of the assessment. For instance, there might be cases where the challenges associated with non-functional requirements may be logically elicited from the text, even though not explicitly stated. To deal with this process, we, therefore, opted for the application of a *fuzzy linguistic approach* [6], which

¹¹Link to *Google Scholar*: <https://scholar.google.com/>

consists of rating each primary study through a continuous variable ranging between 0 and 1. In particular, the scores were assigned as follows:

- 0 => **No**
- 0.1-0.3 => **Rarely**
- 0.4-0.6 => **Partly**
- 0.7-0.9 => **Mostly**
- 1 => **Yes**

In other terms, for each qualitative question, the primary studies were assigned a value reporting how explicit and clear the content was in that respect. At the end of the evaluation conducted for each question, the total merit of a primary study was computed as follows:

- 0 => **No**
- 0.1-1.5 => **Rarely**
- 1.6-2.8 => **Partly**
- 2.9-3.6 => **Mostly**
- 3.7+ => **Yes**

To be finally accepted as part of our systematic analysis, the primary study should have obtained a final score of more than 1.6, i.e., it should have partly specified the required pieces of information to address the research questions of the study.

3.6 Design of the Data Extraction Form

As a final step of the research method applied to address the goals of our study, we designed the data extraction form, namely the specification of the pieces of information to collect when addressing our research questions. Table 4 summarizes the data collected, reporting (i) the dimension the attribute group referred to; (ii) the scope where the data has been used; (iii) the description of the dimension considered; and (iv) the specific attributes considered. As shown in the table, we collected six main categories of information. At first, we identified pieces of information that might help statistically describe our sample in terms of bibliometrics, e.g., publication year and venues: we used the knowledge acquired to describe the trends in terms of publication, the most relevant venues accepting research papers on non-functional requirements of ML-enabled systems, and so on. Besides these meta-data, we then collected and stored information that may be directly connected to the specific research questions posed in our study and that, therefore, was used in the context of the data analysis and reporting process.

3.7 Execution of the Research Methods

Figure 1 overviews the outputs of the execution of the research methods designed in the previous sections. The systematic literature review search was first conducted by the first author of the paper on October 25, 2022. As summarized in the figure, the application of the search query produced a total amount of 2,564 hits. As expected, most of these hits came from *Scopus* (1,973), with *ACM Digital Library* and *IEEEExplore* reporting a lower amount of results (552 and 39, respectively).

The first author of the paper manually collected all the relevant information on the resources identified and stored them within a sheet that was later shared with the second author. Afterward, the first author proceeded with the application of exclusion and inclusion criteria: the former led to filtering out 1,025 research results, while the latter had a total of 1,013. The very limited amount of relevant papers identified after the application of the guidelines by Kitchenham et al. [39] led the two authors to open a discussion about the method defined and the implications that such an outcome might have had on the completeness of the search. In particular, the two authors scheduled two meetings of one hour each where they jointly reviewed the activities performed in an effort to understand the motivations behind the limited amount of resources retrieved. As a result of this fine-grained investigation, they could draw two main observations:

Table 4. Data extraction Form.

Dimension	Scope	Description	Attribute Collected
Paper Information	Bibliometrics	This component includes general information and criteria used to assess the quality of the selected studies.	Seed set or Systematic Literature Review
			Year of publication
			Accepted Score
			Journal or a conference.
Terms concerning non-functional requirements	RQ ₁	This component encompasses all the keywords that make it possible to describe non-functional requirements, both those that already exist in the literature and new non-functional requirements that emerged from this study.	Non-Functional Requirements
Challenges of SE approaches for ML-enabled systems	RQ ₂	This component contains a list of challenges explicitly stated in primary studies and any future challenges and problems emerging from these studies. This information will help us identify areas where further research is needed to improve SE approaches for ML-enabled systems and improve the existing ones.	Challenges
			New Challenges Emerged
Domains in which non-functional requirements are studied	RQ ₃	This component is similar to row 2 but contains all the domains in which non-functional requirements have been studied and found to be problematic. This will make it possible to diversify the impact of AI according to context and to identify the least researched areas.	Domain of Non-functional Requirements
Data-driven terms	RQ ₃	The term data-driven includes all the techniques and technologies used on the datasets before training the models and the ML activities for each dataset the researchers performed.	Datasets
			Task performed with the dataset
			Technologies for testing and analyzing the datasets
			Pre-processing techniques
SE approaches for ML-enabled systems	RQ ₃	This component contains the knowledge areas most suitable for SE approaches for ML-enabled systems. We will collect and classify SE approaches according to different domains of ML systems.	Tasks for each domain
			Architectures of models.
			Type of ML systems
			Tools, Frameworks, and Methodologies to optimize Non-Functional Requirements during and after training.

- (1) The vast majority of the hits were found to be irrelevant to the goals of the study because they mentioned terms connected to non-functional requirements or machine learning either as part of the background or the related work, hence not discussing the challenges posed by non-functional requirements in the context of ML-enabled systems nor proposing automated approaches to cope with them;
- (2) The few relevant hits which passed the inclusion criteria did not only explicitly mention the terms of our search string but also discussed the fine-grained requirements considered.

Based on these observations, the two authors realized the need for boosting the traditional search process with additional steps. While they first considered the re-execution of the systematic search through a search string designed to include all non-functional requirements classified by researchers so far, this would have led to three critical issues. First, the answer to **RQ₁** would have been biased by design: we would have indeed introduced ourselves to the set of non-functional requirements to look for rather than letting them emerge from the analysis of the available literature. Second, the search string would have included a number of *AND/OR* conditions that might have increased noise, leading to the retrieval of several resources not connected to SE4AI research. Last but not least, such an increased noise would have led to the identification of a prohibitively expensive amount of resources to analyze—Habibullah et al. [29] estimated over 200k hits for a search string including all the non-functional requirements of ML-enabled systems classified in literature.

To cope with these challenges, we opted for complementing the systematic literature search with the seed set identification and the snowballing process. Since our goal was that to analyze

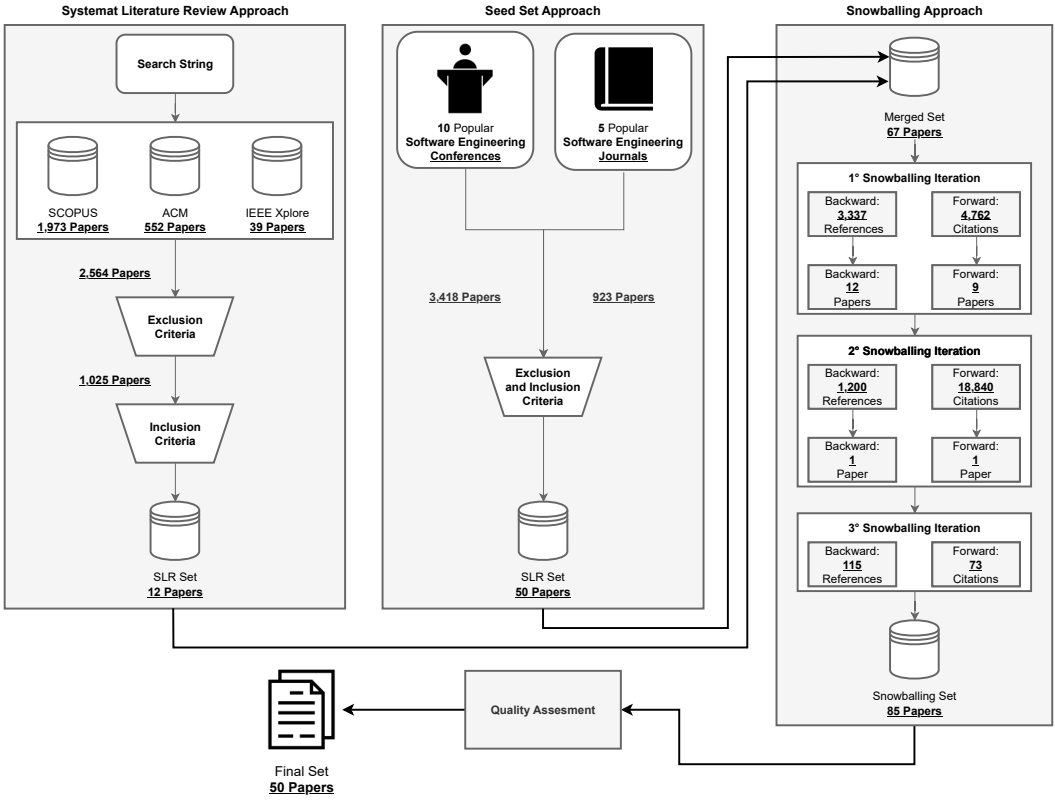


Fig. 1. Overview of the process for selecting papers.

the literature available in the field of SE4AI on non-functional requirements, we then proceeded with the analysis of the papers published to top-tier software engineering venues. The first author of the paper looked for those resources, extracting information concerned to 3,418 papers from conferences and 923 papers from journals. After applying the exclusion and inclusion criteria, 50 unique new resources were deemed relevant, 45 from conferences and 5 from journals.

Before proceeding with the next steps, the second author of the paper reviewed the activities conducted so far. Also, two one-hour meetings were scheduled to discuss the results achieved. The meetings allowed us to acknowledge the soundness of the decision made when opting for a seed search. Hence, the 62 relevant resources acquired so far were taken as input for the snowballing step. Also in this case, the first author was responsible for conducting the search process: by iterating multiple times, both backward and forward, the author was able to identify an additional set of 23 primary studies. Finally, the total amount of 85 papers identified through the three-step search process was considered within the scope of the quality assessment. Given the criticality of this step, this was jointly conducted by the authors: 50 primary studies finally contributed to our study.

To make the reader aware of the effort required to conduct the search/analysis process and contribute to the transparency/replicability of our work, we also estimated the number of person-hours invested in the work. As for the first author, the estimation is about 480 person-hours; as for the second, it is about 120 person-hours.

Reproducibility and Replicability of Our Work.

Our online appendix reports all the fine-grained steps described above. The interested reader may use the appendix to replicate our work or build on top of our findings to create additional knowledge on non-functional requirements of ML-enabled systems: [58]

4 ANALYSIS OF THE RESULTS

In this section, we report quantitative and qualitative insights coming from the data extraction and analysis phase. We first analyze bibliometric data to describe the sample considered in our research. Afterward, we address the specific research questions of the study.

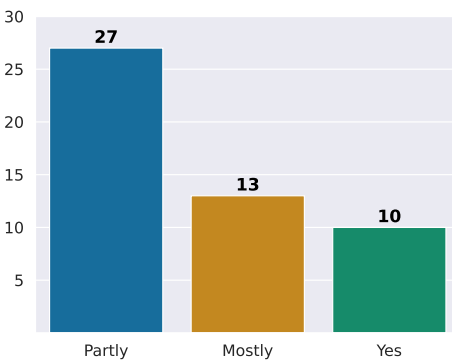


Fig. 2. Results of the quality assessment.

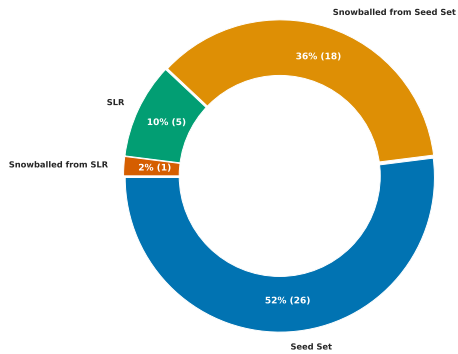


Fig. 3. Papers per publication venue.

4.1 Bibliometrics

Figure 2 overviews the results of the quality assessment procedure. We finally included a total of 50 primary studies. Most of them (27, 54%) received an overall quality score of “Partly”, i.e., they specified the pieces of information required to address our research questions mostly implicitly, yet giving us a chance to elicit them satisfactorily. The other 23 primary studies were more explicit and, indeed, reached an overall quality score of “Mostly” or “Yes”. The primary studies were mostly retrieved through the seed set search (26, 52%), while the other 18 resources (36%) were identified by snowballing the seed primary studies identified. Only 5 papers were retrieved through the systematic literature search and only one additional resource could be identified through the snowballing process conducted on the set of papers identified with the systematic search. These considerations further justified our choice of complementing the traditional systematic literature search with additional instruments.

More interesting were the insights coming from the publication venues, which are shown in Figure 4. We observed that the vast majority of the primary studies (44, 88%) appeared in conference proceedings, with a limited amount of resources published in journals (6, 12%). This finding seems to suggest that the research on non-functional requirements of ML-enabled systems is still at its early stage, with researchers interested in discussing recent advances in venues that allow discussion and interaction with the research community. On a similar note, the result may stimulate young researchers to enter this growing research area. The analysis of the amount of papers published on the matter per year confirmed the early nature of the research area (Figure 5). According to

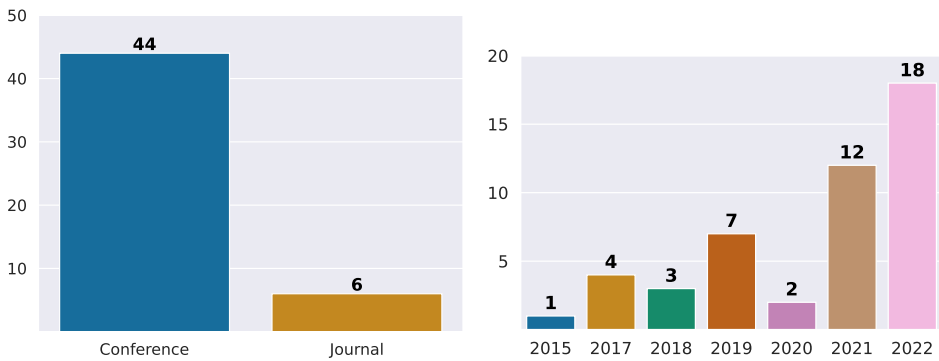


Fig. 4. Papers published per publication venue. Fig. 5. Papers published on the matter over time.

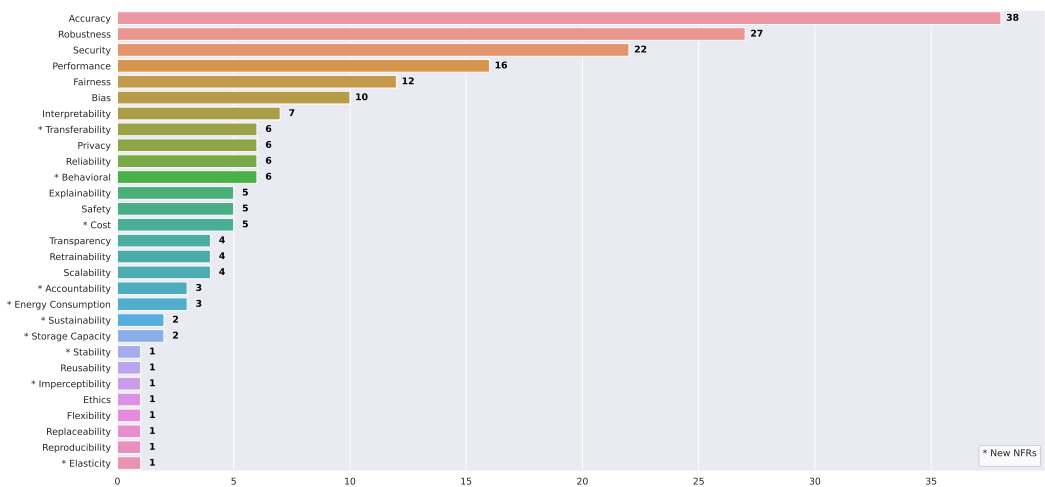


Fig. 6. Source-wise Distribution of Non-functional Requirements in ML-Enabled Systems.

our findings, only a few papers were published from 2015 to 2020, while a notably increasing trend could be found in the last two years. Performing a systematic synthesis of the knowledge collected so far could provide a relevant boost to the research activities that will be performed in the next years: our work indeed identifies multiple implications and challenges that the software engineering research community will be called to address.

4.2 RQ₁ - What are the non-functional requirements of ML-enabled software considered by researchers?

The first goal of our work was to classify the non-functional requirements of machine learning-enabled systems. The primary studies considered were first analyzed in an effort to elicit the non-functional attributes that they aimed to address. Afterward, we provided the extracted non-functional attributes with a (1) name, (2) a description, and (3) a reference class, i.e., a high-level class that may be used to cluster together multiple non-functional requirements based on similarity, i.e., non-functional requirements sharing similar purposes. For instance, requirements such as “*privacy*” and “*security*” were both extracted and mapped onto the “*Resiliency*” class, as they both referred to

aspects making an ML-enabled system resilient to unexpected events arising in operation, including external attacks. Whenever possible, i.e., when considering the papers that received a quality score of “Yes”, the extraction was pretty straightforward as these resources explicitly referred to the non-functional requirements considered; as such, they could be easily extracted and mapped onto known classes of non-functional requirements or onto new classes according to the conceptual similarity between the non-functional requirement analyzed and the set of classes identified. In the other cases, relevant terms connected to non-functional aspects were extracted, elaborated, and only later mapped onto the corresponding class of non-functional requirements. For example, the primary study [S1] reported terms like “*memory problems*” and “*battery drain*”, which we used to interpret the context of the study and assigned the non-functional aspect to the class entitled “*Sustainability*”. As an output of the process, we were able to develop a taxonomy of non-functional attributes of ML-enabled systems.

It is worth pointing out that the names assigned to the classes, the description of each non-functional requirement, and the classification exercise as a whole, were informed by different sources of the existing body of knowledge, i.e., by the primary studies collected in this paper, other resources of the state of the art not directly related to our research goals, and the *Systems & Software Quality Requirements & Evaluation* standard (ISO/EIC 25010). In particular, the process was initially conducted by the first author of the paper, who (1) analyzed the primary studies to elicit the non-functional attributes considered and (2) provided a preliminary version of the taxonomy. The author exploited the existing body of knowledge in an effort to homogenize the definitions and classification; e.g., he favored the assignment of well-established class names whenever possible in an effort to provide a more comprehensive and usable taxonomy. The preliminary taxonomy was later the subject of evaluation. The second author joined the process at this stage, and a discussion was opened on the taxonomy produced. The two authors discussed about the consistency of the classification performed, other than on the names and descriptions assigned. Whenever needed, they modified the taxonomy—this happened in four cases, in which some non-functional requirements were grouped differently, and the names assigned to the classes were modified.

In the first place, as a result of this procedure we could identify a total amount of 29 non-functional requirements. These were: *accuracy* [S2–S39], *robustness* [S2–S7, S9, S10, S12, S19, S20, S22–S24, S27–S31, S40–S47], *security* [S2, S3, S7, S9, S12, S14, S19, S22–S24, S27, S31–S33, S36, S40, S41, S43–S45, S47], *performance* [S1, S10, S13, S15–S17, S28–S30, S32, S37, S39, S43–S45, S48], *fairness* [S13, S15, S17, S21, S26, S33–S35, S46, S48–S50], *bias* [S13, S15, S17, S21, S26, S34, S35, S46, S48, S50], *interpretability* [S14, S17, S27, S33, S38, S39, S44], *transferability* [S24, S31, S40, S42, S43, S45], *privacy* [S8, S13, S32, S33, S36, S47], *reliability* [S20, S27, S28, S33, S44, S47], *behavioral* [S7, S14, S20, S28, S44, S46], *explainability* [S9, S21, S38, S39, S46], *safety* [S6, S7, S14, S24, S42], *transparency* [S34, S39, S46, S50], *cost* [S18, S22, S33, S37], *retrainability* [S3, S5, S23, S47], *scalability* [S25, S27, S41, S42], *accountability* [S34, S49, S50], *energy consumption* [S1, S18, S37], *sustainability* [S18, S25], *storage capacity* [S1, S37], *stability* [S27], *reusability* [S25], *imperceptibility* [S6], *ethics* [S34], *flexibility* [S47], *replaceability* [S25], *reproducibility* [S16], and *elasticity* [S33]. Interestingly, our systematic exercise could identify not only non-functional requirements that emerged already in previous studies [28, 29, 33], but also additional categories that were not previously pointed out, i.e., *transferability*, *cost*, *accountability*, *energy consumption*, *sustainability*, *storage capacity*, *stability*, *behavioral*, *imperceptibility*, and *elasticity*.

Figure 6 depicts the amount of primary studies that aimed at considering each of the 29 non-functional requirements identified. As somehow expected, non-functional requirements such as *accuracy* and *robustness* of ML-enabled systems were those more frequently considered (38 and 27 times, respectively): these two aspects are likely the ones considered as recurrent and fundamental features of any machine learning model. Nonetheless, aspects connected to *security* (21), *performance*

(16), *fairness* (12), and *bias* (10) seem to be growing research areas. On the contrary, multiple aspects were little explored, e.g., *reusability* or *scalability*, possibly indicating emerging research themes that would be worth to further exploring in the future.

Table 5. Non-Functional Requirements in ML-enabled systems.

Cluster	NFRs	Definition
Accuracy	Accuracy	The degree to which a model's predictions match the actual values.
Efficiency	Performance	The ability of a system to perform actions within defined time or throughput bounds.
	Storage Capacity	The amount of space required to store the model and any associated data.
	Stability	The ability of the model to produce consistent results over time, regardless of any changes to the input data or model parameters.
	Scalability	The degree of the model to handle increasing amounts of data, labels, and resources while maintaining good performance.
Maintainability	Replaceability	The degree to which a model can be replaced or substituted with another model without significant changes to the system.
	Retrainability	The degree to which a model can be retrained on new data without significant performance loss.
	Reproducibility	The degree to which a model's results can be reproduced by others using different software or hardware.
	Transferability	The degree to which a model trained on one data set can be applied to another with similar characteristics.
	Reusability	The degree to which a model can be reused in different applications or contexts.
	Elasticity	How ML inferences deal with dynamic data volume and velocity changes.
Resiliency	Security	The degree to which a model and its associated data are protected against unauthorized access, modification, or theft.
	Safety	The degree to which a model and its outcomes are safe for humans and the environment.
	Privacy	The degree to which a model and its associated data protect individuals' privacy rights and comply with data protection regulations.
	Robustness	The degree to which a model can perform accurately and reliably in the face of errors, noise, or adversarial attacks.
	Reliability	The degree to which a model can be trusted to perform accurately and reliably over time, under different conditions.
	Behavioral	The degree to which a model's outcomes align with requirements and expectations.
	Flexibility	The degree to which a model can adapt to input data or environment changes without significant performance degradation.
Sustainability	Fairness	The degree to which a model produces unbiased predictions and decision-making outcomes across different groups of individuals.
	Bias	The degree to which a model produces unfair predictions and decision-making outcomes that disproportionately favor or disadvantage certain groups of individuals.
	Ethics	The degree to which a model's design, development, and deployment adhere to ethical principles and standards.
	Accountability	The degree to which individuals or organizations are held responsible for the actions of the model and its outcomes.
	Cost	The resources required to train, deploy, and inference of the model.
	Energy Consumption	The amount of energy required to train and inference of the model and its impact on the systems.
Usability	Interpretability	The degree to which a model's predictions and decision-making process can be explained in terms of causality or human-understandable concepts.
	Imperceptibility	The system's ability to produce outputs that are indistinguishable from what a human would produce in the same scenario.
	Explainability	The degree to which a model's predictions can be explained and understood by humans.
	Transparency	The degree to which a model's inner workings and decision-making process can be understood and evaluated by humans.

The classification exercise led to the identification of six main classes of non-functional requirements: Table 5 reports, for each class, the set of non-functional requirements belonging to the class, along with their definition. More specifically, we identified the following classes:

Accuracy. The first class contained only the accuracy requirement, namely the degree to which a model's predictions match the actual values. The class was made isolated because of two main

reasons: (1) accuracy represents the key feature to optimize by machine learning-enabled; (2) the non-functional requirement cannot be conceptually compared to any other, being “unique”.

Efficiency. This was concerned with the overall performance and effectiveness of machine learning-enabled systems. The ISO/IEC 25010 standard includes performance efficiency as one of its main characteristics, encompassing attributes such as time behavior, resource utilization, and storage capacity. As such, we exploited the same reasoning to cluster non-functional requirements such as *performance*, *storage capacity*, *stability*, and *scalability*, under the “Efficiency” class.

Maintainability. Maintainability is chosen as a cluster to represent *replaceability*, *retrainability*, *reproducibility*, *transferability*, *reusability*, and *elasticity* because it emphasizes the ability of a system to adapt and evolve over time. All these non-functional requirements are critical to ensure that the system can be maintained and updated as needed and that it can be easily adapted to new use cases or environments. By prioritizing maintainability, designers, and developers can create reliable, efficient, adaptable, flexible, and sustainable systems over the long term.

Resiliency. Resilience was chosen as a cluster to represent *robustness*, *reliability*, *behavioral flexibility*, *security*, *safety*, and *privacy* because it emphasizes the ability of a system to adapt and recover from adverse events while maintaining its functionality and performance. Robustness, reliability, and behavioral flexibility are essential to ensure the system can operate in various conditions and remain responsive to changing circumstances. Security, reliability, and confidentiality are also critical to ensure the system protects users and stakeholders from harm, including threats to their physical safety, personal information, and data confidentiality. By prioritizing resilience, designers and developers can build reliable and secure systems that can withstand disruptions and threats while maintaining their core functions and services.

Sustainability. Sustainability is chosen as a cluster to represent economic, social, and environmental aspects. It ensures that resources are used efficiently and effectively, reducing the environmental impact, *energy consumption* and financial *costs* associated with producing the model and making inferences. Additionally, sustainability is chosen to represent *fairness*, *ethics*, *bias*, and *accountability* because these aspects are critical for ensuring that the system benefits all stakeholders, including marginalized and vulnerable communities, and operates in a way that aligns with ethical principles and values. Overall, sustainability is a guiding principle for designing and developing efficient, cost-effective but also equitable, ethical, and accountable systems.

Usability. The usability class is similar to the one defined by Habibullah et al. [28]. More particularly, this class was chosen as the cluster to represent *interpretability*, *imperceptibility*, *explainability*, and *transparency*: it is concerned with ensuring that a system is easy to use and understand for end users, reducing the cognitive load required to interpret the model’s behavior, and making it more transparent and explainable.

Researchers can use the taxonomy built in the context of our work to have a comprehensive mapping of the relevant non-functional requirements to optimize when developing machine learning-enabled systems, other than understanding the quality and efficiency aspects practitioners should focus on when releasing machine learning models.

🔗 **Answer to RQ₁.** As a result of RQ₁, we could identified a total of 29 *relevant non-functional attributes*, classified according to *six main categories*. The resulting taxonomy maps the current knowledge on the matter and provides insights on the non-functional attributes that researchers and practitioners may want to focus on when proposing novel optimization approaches or developing machine learning-enabled systems respectively.

4.3 RQ₂ - What are the challenges of dealing with non-functional requirements of ML-enabled software?

The second research question concerned with the identification of the challenges faced in dealing with the non-functional requirements of machine learning-enabled software. To reach such an objective, the primary studies were first analyzed to gather information on the specific domain(s) targeted: this information might have potentially shed lights on the contexts where the non-functional requirements identified in RQ₁ should be more carefully taken into account. In the second place, we elicited the main problem that primary studies considered: in this way, we could elaborate on the motivations making non-functional requirements hard to treat. Finally, we identified the main challenges to address those problems: in doing so, we focused on the software engineering perspective, hence attempting to produce open challenges that our research community is called to address through further investigations. For the sake of readability, we split the discussion of the findings achieved in the context of RQ₂ by step conducted during the analysis phase.

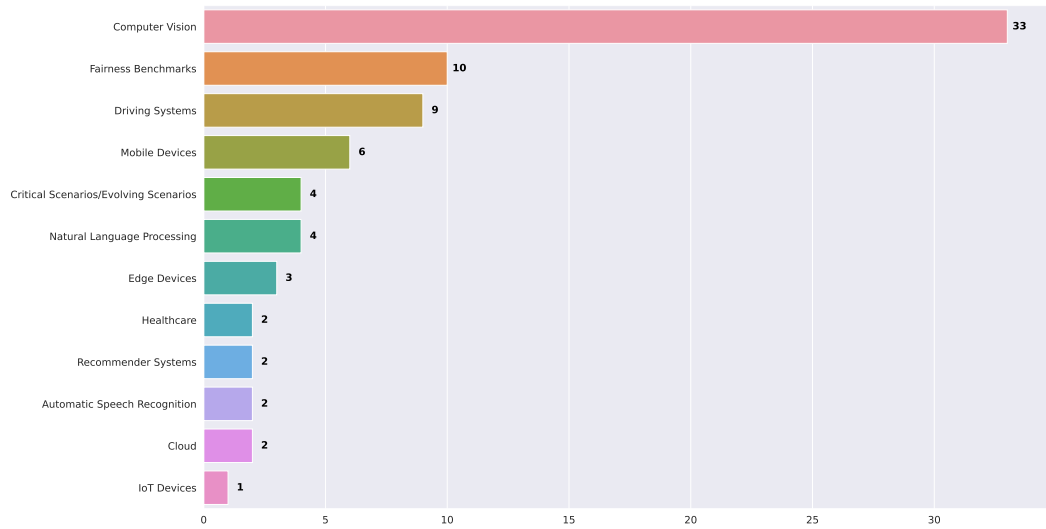


Fig. 7. Distribution of Non-Functional Requirements across domains in ML-enabled systems.

4.3.1 On the domains where non-functional requirements issues manifest themselves. Figure 7 overviews the amount of primary studies targeting each of the application domains extracted. As shown, the most frequent are ‘Computer Vision’ (33), ‘Fairness Benchmarks’ (10), and ‘Driving Systems’ (9), while other, less targeted domains pertain to emerging technologies, e.g., ‘IoT Devices’ or ‘Automatic Speech Recognition’. On the one hand, these insights let arise some key application domains that researchers have been analyzing in the recent past. On the other hand, they also let arise contexts where further research might be worth to be focused.

The much larger amount of primary studies targeting ‘Computer Vision’ may also indicate the critical nature of such a domain with respect to the management of non-functional requirements. This may be due to the nature of the inputs that the machine learning-enabled systems should consider, i.e., images or videos, which may be the subject of multiple concerns such as accuracy, security, ethics, and privacy—which may potentially emphasize the need for novel methods to process images and/or deal with non-functional attributes in the field. Nonetheless, It is worth

remarking that 16 studies did not precisely describe the specific application domain considered—as such, they were deemed too generic to be mapped onto those reported in Figure 7. Nonetheless, the insights we were able to extract may already suggest where previous research invested time, other than the application domains where further research might focus on.

4.3.2 On the challenges to face when dealing with non-functional requirements of machine learning-enabled systems. Our work identified more than twenty software engineering challenges for our research community. First and foremost, it is worth pointing out that most of the challenges identified pertained to the use/integration of neural networks within more complex software systems. This was somehow expected, as deep learning has become the most widely employed form of machine learning to empower the capabilities of traditional software systems [26]. In the second place, it is important to remark that accuracy represents a cross-cutting concern, i.e., all the challenges identified have implications for the accuracy of machine learning-enabled systems. For this reason, we preferred to discuss accuracy while presenting the challenges connected to the other classes of non-functional requirements identified in **RQ₁**.

Q Efficiency (E): In terms of overall performance and effectiveness of machine learning-enabled systems, we could elicit three major challenges which are connected to internal errors, high latency, and memory issues. Specifically:

- **Challenge E.1 - Dealing with Internal Errors.** Neural networks can produce erroneous outputs as a consequence of internal errors due to *incorrect parameters*, *incorrect weight values*, *uncovered root causes*, and *incorrect manual labeling*. Those internal errors may significantly impact the performance of machine learning-enabled systems, leading to reduced accuracy, slower inference times, and increased energy consumption [S1, S3, S37]. For example, incorrect weight values or parameters can cause the network to converge slowly or get stuck in suboptimal solutions, resulting in poor performance. Dealing with internal errors has two main connotations such as tracking errors in deep neural network models and addressing them. Our findings suggest that research in terms of *bug localization*, *performance monitoring*, and *program repair* might support the challenges identified.
- **Challenge E.2 - Dealing with High Latency.** Literature reported that multiple design choices related to *hyper-parameter tuning*, *model optimization*, and *model architecture* can affect the overall efficiency of neural networks, even though those decisions might sometimes lead to increased accuracy [S37]. In other terms, the definition of neural networks should be considered as an optimization problem, where designers look for the best trade-off between performance and other quality measures and carry out preliminary studies to find the best configurations to employ within the specific use case. As such, researchers in the field of *software analytics*, *software architecture*, and *software quality and optimization* might help address this challenge by means of analytical and code quality instruments aiming at finding the right compromise between accuracy and efficiency.
- **Challenge E.3 - Dealing with Space.** A third critical challenge arising from our review consists of the space required by machine learning models and, particularly, by neural network-based solutions. These may indeed present memory issues due to their large size, which has been particularly relevant in contexts like edge computing and IoT. Deploying large models on such devices can result in memory constraints affecting system performance [S1, S37]. In response to this challenge, designers should be able to apply policies and strategies to build reduced models before deploying them on devices. These strategies may mitigate memory issues and improve the overall performance of machine learning-enabled systems. In this respect, our findings call for further research on the emerging field of *tiny machine learning*

[47], that is, the combination of hardware, algorithms, and software that support on-device sensor data analytics at low power. While some initial effort has been made in the artificial intelligence research community (e.g., [22, 77]), we point out a lack of knowledge of the software engineering side of the matter.

The three challenges identified were discussed in all the application domains requiring the definition of large models, i.e., ‘*Computer Vision*’, ‘*Driving Systems*’, ‘*Natural Language Processing*’, ‘*Mobile Devices*’ and ‘*IoT Devices*’. This was somehow expected, as scalability concerns become evident in environments requiring high computational demands.

Q Maintainability (M): When it turns to the ability of machine learning-enabled systems to be adapted and evolved over time, we could elicit two main challenges connected to model reproducibility and model decomposition and reuse.

- **Challenge M.1 - Increasing Model Reproducibility.** Machine learning-enabled systems based on deep learning models are known to be complex and difficult to reproduce accurately. One of the primary challenges in reproducing deep learning models concerns with randomness, which can affect the behavior of the algorithms used. Additionally, hardware non-determinism, such as that present in graphics processing units (GPUs), can further complicate the reproducibility of deep learning models [S16]. Our findings seem, therefore, to suggest the need for mechanisms that could support the *verification* of deep learning models, for instance, by injecting elements of randomness and non-determinism to check how the system may behave in those circumstances.
- **Challenge M.2 - Increasing Model Decomposition and Reuse.** A critical challenge when building and improving machine learning-enabled systems based on deep learning models is the need to reuse parts of previously constructed models. This can be difficult, for instance, when attempting to replace potentially defective parts with others. A possible approach concerns with the identification of the parts responsible for each output class or module in the models: this would allow to selectively reuse or replace only the desired output classes to build the model. This challenge is strictly connected to the properties of randomness and non-determinism already discussed in the previous point. Indeed, increasing decomposition and reuse requires a proper understanding of the potential sources of variability and non-determinism in the system’s software and hardware components. Therefore, improvements in terms of reproducibility may also lead to more reusable models [S16, S25]. On the one hand, our findings reinforce the need for research in terms of *verification*. On the other hand, the challenge of identifying reusable components seems to call for research in terms of *program slicing* and *refactoring*, which might support machine learning engineers to properly extract parts of the models for reusability purposes.

Challenges connected to maintainability were found to be pervasive in all domains, hence representing concerns that may impact any kind of machine learning-enabled system.

Q Resiliency (R): Improving the universal robustness and security of machine learning models present key issues [S12]. This was the class of non-functional requirements where we identified more challenges (8); these were connected to various aspects such as adversarial attacks, theft of model and intellectual property, attack and defense of models, repairing internal models, post-deploy issues, preserving privacy, and vulnerability transferability.

- **Challenge R.1 - Resilience to Adversarial Attacks.** Machine learning models, and more particularly deep neural networks, are vulnerable to the so-called *adversarial attacks* [55, 81], namely attacks that manipulate the input data in a way that causes the neural network to produce an incorrect output and significantly drop accuracy. Different types of adversarial

attacks can be targeted or untargeted and can be carried out through various techniques such as *adversarial examples* [81], *adversarial perturbation* [9], *poisoned data* [34], and *backdoor samples* [27]. These types of attacks can negatively affect the overall robustness of the model and decrease its accuracy. As such, a key challenge is represented by the resilience to adversarial attacks [S2, S40, S43]. More specifically, existing literature highlighted the need for instruments able to design perturbations that may properly generate adversarial examples for different machine learning models, e.g., rotations, smoothing, and erosion operations in machine learning-enabled systems targeting computer vision problems [S4, S6, S10, S42]. Those instruments might support practitioners in terms of an improved comprehension of how models can be affected by vulnerabilities, other than through improved methods to increase appropriate defenses against adversarial attacks. Hence, our findings suggest additional *verification and validation* research efforts: indeed, while the security perspective has been often targeted by the artificial intelligence community through the definition of new algorithms and techniques [46, 86], software engineering research might be complementary and propose novel verification and validation approaches, which may improve the policies and strategies employed by practitioners when testing for security.

- **Challenge R.2 - Exploiting the Sensibility of Adversarial Attacks.** A complementary perspective is concerned with the sensibility that certain adversarial attacks may have to input distortions and how such a sensibility can be exploited to improve the security of machine learning models. For example, in computer vision applications, attacks made through non-additive noise or geometric morphing may cause serious security concerns, yet those filters are sensitive to input distortions, meaning that automatic correction instruments may potentially distort the malicious inputs to mitigate their effects on the machine learning model [S10, S40, S41]. Our findings, therefore, point out the need for techniques that may handle perturbations that not only occur naturally in the physical environment but could be maliciously generated by adversary attacks. The perturbations are typically stealthy and undetectable, which poses further challenges for systems based on deep neural networks [S4]: as such, characterizing them would be the first step for *empirical software engineering* and *automatic program repair* researchers.
- **Challenge R.3 - Security Verification of Pre-Trained Models.** Pre-trained models can be beneficial, as they reduce the computational burden of training complex deep learning models through transferability. However, pre-trained models may be vulnerable to attacks due to the displacement of the dataset and the potential for malicious code or exploitation [S43, S45]. On the one hand, assessing the security risks of these models still represent a key challenge to face through the definition of software analytics instruments able to provide practitioners with security insights and best practices in selecting the most appropriate pre-trained model to use to avoid security concerns. On the other hand, existing literature [S19, S43, S45] let also emerge additional challenges targeting the definition of design practices that may allow to develop and retrain security-aware fine-tuned models. Our findings, therefore, suggest that researchers in the field of *software analytics*, *software quality*, and *verification and validation* might contribute to addressing the challenges in this set.
- **Challenge R.4 - Resilience to Intellectual Property Theft.** Machine learning-enabled systems based on neural networks suffer from data breaches and unauthorized access to sensitive data due to training attacks [84], i.e., malicious attacks on training data. According to the existing literature, only a little knowledge is available on how to secure machine learning models against those types of attacks. Greater attention should be paid to strengthening security protocols [S32, S43]. While this mostly calls the attention of researchers in the field

of networks and security, the software engineering research community might still contribute through the definition of more security best practices able to protect models from intellectual property theft, especially in edge devices and IoT systems [95].

- **Challenge R.5 - Diagnosing the Internal Behavior of Models.** Understanding the root cause of anomalous behaviors of deep neural networks and how to fix them represent critical aspects to further elaborate. On the one hand, evaluating the reliability implications of small sets of weights assigned to the network may support practitioners toward an improved understanding of how they work. In this respect, the definition of exploration, interpretability, and explainability methods may lead to substantial advances in terms of diagnosis of failures and misclassification, other than of security concerns [S9, S46]—Ji et al. [S28] also called for novel local models to uncover the root causes of deep neural network failures. On the other hand, existing literature also pointed out the need for novel assessment metrics that, besides accuracy, can provide actionable insights into the overall robustness of the model [S11, S27]. As such, our findings suggest multiple avenues for software engineering researchers in the field of *fault localization*, *explainable AI*, *software metrics*, and *automatic program repair*.
- **Challenge R.6 - Optimal Post-Deployment Simulation.** While some threats to resiliency might be managed at the development time, additional challenges arise at the deployment stage. In particular, machine learning models could not work as expected when deployed. The problem has been mainly pointed out in the context of automatic speech recognition [S28, S29], yet it may affect any kind of machine learning solution. The main challenge is related to properly understanding how the model would work in a real-case scenario. Besides software testing [72], literature identified post-deployment simulations as a complementary instrument that might be worth to further investigate: this refers to the definition of agent-based models that may (i) simulate the environment where the machine learning-enabled system would work and (ii) verify the system against a large variety of simulated inputs. This is, again, something that may catch the attention of researchers in the field of *verification and validation* and *machine learning for software engineering*.
- **Challenge R.7 - Preserving Privacy in Machine Learning-Enabled Systems.** The primary studies identified the challenge of developing privacy-preserving deep neural networks and machine learning systems, which may allow algorithms to run securely on distributed data without compromising sensitive information about the subjects of the data, leaving users with the ability to delete their data at any time. This would require the definition of strategies to train models without sacrificing accuracy while ensuring the protection of private data [S8, S36]. As such, this challenge targets multiple software engineering fields, from *software quality* to *software architecture*, which may deepen the current knowledge on data encryption, anonymization techniques, federated learning, and differential privacy.
- **Challenge R.8 - Improving the Generalizability of Existing Automated Frameworks.** Multiple primary studies identified limitations and challenges related to the automated support provided by existing frameworks. Challenges in this category mainly concern with their generalizability. For instance, PLUM [S5], a recommendation system to prioritize model repair strategies, is limited to the analysis of deep learning models and does not provide support for different learning tasks. Similarly, other approaches [S7, S20] can only assist practitioners when developing specific types of neural networks. As such, we also identified technological and empirical challenges for our research community, which is called to develop and experiment with automated solutions by keeping generalizability into account.

Challenges in this category were mainly considered in the domains of ‘*Computer Vision*’, ‘*Driving Systems*’, ‘*Mobile Devices*’, ‘*Edge Devices*’, and ‘*IoT Devices*’, i.e., those more intensively threatened by security issues. Nonetheless, we observed a rising interest toward ‘*Automatic Speech Recognition*’—this is likely due to the growing research on large language models [18].

Q Sustainability (S): As for the challenges in this category, we could observe that most of the available literature focused on ethics and fairness, somehow neglecting other perspectives of sustainability. More particularly, we identified seven main challenges connected to algorithmic discrimination, model accountability, fairness metrics, low-quality datasets, energy cost, energy and performance aware trade-offs, and generalizability of existing solutions.

- **Challenge S.1 - Dealing with Algorithmic Discrimination.** Improving performance, accuracy, and fairness simultaneously is currently the main challenge for researchers [S13, S15]. More specifically, the challenge is to study and mitigate the impact of disparate results, offensive labeling, and uneven algorithmic error rates in data-driven applications. In addition, the primary studies investigated highlighted the relevance of dealing with algorithmic discrimination in the context of the pre-processing phase in an effort to improve the trade-off between fairness and performance in ML software. While the software engineering research community has already contributed to addressing this challenge (e.g., [S46, S49]), our findings suggest that further research in terms of *software analytics* and *verification and validation* might be worthwhile.
- **Challenge S.2 - Model Accountability:** The term ‘accountability’ refers to the model’s ability to provide clear explanations of its decisions, its transparency in how it has been trained and makes decisions, and its ability to allow users to provide feedback or challenge its decisions. The primary studies considered in our work remarked that more effort should be given to this aspect to allow models to be deployed with an eye on ethics and fairness. In this respect, major challenges pertain to the definition of *verification and validation* instruments through which practitioners can verify how accountable their systems actually are [S49, S50]. For instance, fairness benchmarks may represent ideal tools to assess the level of accountability of machine learning models.
- **Challenge S.3 - Fairness Analytics.** Another critical challenge for social sustainability is related to fairness analytics, that is, the definition of metric toolkits that may support practitioners in diagnosing fairness at the development time. However, fairness should not be considered as a non-functional requirement per se, but rather be part of trade-off analyses. More specifically, existing literature called for novel metrics able to provide insights into the compromise between fairness and accuracy [S26], fairness and privacy [S8], and fairness and accountability [S49]. As such, the challenge is represented by the diagnosis and multi-objective optimization of data-driven applications in multiple development stages, from requirements engineering to verification and validation [S49].
- **Challenge S.4 - Improving Sustainability Benchmarks.** Current literature [S50] also highlighted the lack of low-quality of benchmark datasets that may assist the creation of sustainable machine-learning applications. These datasets should indeed be devised by enabling cross-domain and cross-sectional analysis. On the one hand, application domains focusing on the image processing would require high-quality datasets allowing automated pose, illumination, and expression (PIE) analysis. On the other hand, datasets presenting large and diverse information on minorities should be devised to allow the more effective training of machine learning systems.
- **Challenge S.5 - Reducing Energy Cost.** One of the most important challenges for deep neural networks is concerned with the reduction of energy and computational costs. Indeed, those

systems have considerable energy and financial costs, which lead to increase CO2 emissions and memory consumption in both the training and post-release phases [S18, S20, S33, S37]. More specifically, energy consumption issues significantly impact edge and IoT devices, which indeed represent critical use cases. The software engineering research community is called to contribute in different manners. First, *empirical software engineering* research is called to assess the impact of different AI containerization strategies on energy consumption and memory utilization. Second, *software quality* and *software architecture* researchers might help address the trade-offs between training accuracy and post-deployment consumption. Last but not least, our findings call for further research on the definition of software engineering practices to develop *tiny machine learning* solutions.

- **Challenge S.6 - Increasing the Practitioner's Awareness of Sustainability.** Our literature work identified the challenge of increasing the developer's awareness with respect to sustainability concerns. This issue is particularly relevant when considering energy consumption, which is a critical concern in today's world. As such, the definition of novel strategies and instruments to make practitioners aware of the trade-offs between performance and sustainability represents a key challenge for our research community [S1, S18]. Once again, more research on tiny machine learning might represent a suitable solution, as it would allow to improve the scalability of models without compromising their overall accuracy. At the same time, the *empirical software engineering* research community might play a key role by understanding the practitioners' needs and practices, other than tailoring (novel) automated approaches on them.
- **Challenge S.7 - Improving the Generalizability of Existing Automated Approaches.** The last challenge in this category concerns with the generalizability of existing approaches that support sustainability analysis of machine learning-enabled systems. Multiple primary studies identified limitations and challenges that the research community should further consider. Similarly to **R.8**, current approaches can only provide support for the development of a limited set of deep neural networks [S17, S21], are built using limited datasets [S34], or are not integrated within MLOps pipelines [S35]. In other terms, the challenge emphasizes the need for novel, more accurate, generalizable, and integrated instruments that may support practitioners throughout the software lifecycle.

Since most of the challenges described pertained to social sustainability, these were investigated in the '*Fairness Benchmarks*' and '*Healthcare*' domains, which are the most demanding in terms of ethics and fairness. However, we observed a growing interest in the '*Cloud*', '*IoT Devices*', and '*Edge Devices*' domains, mostly due to the increasing efforts on energy consumption matters.

Q Usability (U): According to our findings, usability is the non-functional requirement for which we could identify fewer challenges. Only a few works targeted this matter, hence suggesting that further research might be worthwhile. Indeed, usability, interpretability, and visualization of both shallow and deep learning models pose significant challenges when it turns to assess quality assurance throughout the software lifecycle. For instance, the lack of interpretability increases the effort required to estimate the soundness of machine learning-enabled systems. Current literature somehow neglected this research angle, especially when considering models operating in critical and evolving scenarios. As such, our findings suggest the urgent need for further research on usability concerns [S14, S37–S39, S44]. Interestingly, the available primary studies did not focus on any specific application domain and proposed general-purpose interpretability models. This further suggests the investigation of domain-specific approaches and methods.

🔗 **Answer to RQ₂.** As a result of RQ₂, we elicited more than 20 software engineering challenges across the categories of non-functional requirements identified in our systematic synthesis work. We methodically untangled the challenges plaguing machine learning-enabled systems, providing insights into the specific research fields interested in those challenges. However, none of the non-functional requirements can be treated alone, as they can significantly influence each other. For this reason, we call for comprehensive analyses and approaches that may assess the impact and implications of multiple non-functional requirements simultaneously.

4.4 RQ₃ - What automated approaches are proposed to deal with non-functional requirements of ML-enabled systems?

With our last research question (RQ₃), we were interested in surveying the automated approaches to handle non-functional requirements of machine learning-enabled systems proposed so far by researchers. On the one hand, this analysis allowed us to collect information on multiple aspects of those approaches, hence providing insights into the current support available and the related limitations. On the other hand, we could also measure the alignment between the classes of non-functional requirements identified and the approaches available in an effort to understand the extent to which the current approaches fulfil the potential practitioner's needs and, at the same time, what are the limitations of the state of the art and the non-functional requirements that researchers should be more closely pay attention to in future research.

To address our goal, we analyzed the primary studies from two main perspectives. As further discussed in the remainder of the section, all the approaches retrieved were based on machine learning: as such, we first analyzed how these machine learning solutions were fed in terms of training datasets and architectural decisions. In the second place, we further deepen our investigation and survey, for each approach, the non-functional requirements addressed and the development phase where it may be applied. By means of these two perspectives, we could basically investigate the design decisions made by researchers when devising machine learning solutions able to address non-functional requirements of machine learning-enabled systems.

4.4.1 On the training decisions behind automated approaches. Tables 6 and 7 report the results achieved when considering how the automated approaches considered where fed. More particularly, Table 6 overviews the datasets retrieved in our study. The datasets are first grouped according to the domain in which the primary study focused. The domains are those resulting from RQ₂, yet we also included a category named 'General Purpose': this was required because some datasets could be used for multiple purposes and tasks. The datasets are then further grouped by the tasks that it allows to experiment with, e.g., text versus speech classification. Finally, the last column of Table 6 provides names and references of the datasets—the reader may observe the missing reference to the 'Contagio/Virustotal' and 'Raw Car Rentals' datasets: they are currently not accessible or have restricted access. At the same time, Table 7 maps the the model architectures employed by the automated approaches, namely the machine learning algorithms exploited when building them.

From Table 6, we could first observe a mismatch between the application domains identified in RQ₂ and those identified here. In the first place, this finding can already provide indications of the comprehensiveness of the automated approaches proposed so far: these only exploited datasets pertaining to a few domains such as 'Computer Vision', 'Fairness Benchmark', 'Natural Language Processing', and 'Automated Speech Recognition', hence possibly neglecting the additional domains identified in our review. At the same time, our finding may be a reflection of a lack of datasets that can be used to comprehensively investigate non-functional requirements of machine learning-enabled systems—our work focused on the datasets exploited by automated approaches rather than

Table 6. Overview of the datasets employed by the automated approaches proposed in the literature.

Domain	Task	Dataset
Computer Vision	Image Classification	CIFAR-10 - CIFAR-100 [41]
		ImageNet [21, 42]
		MNIST [45]
		SVHN [65]
		Fashion MNIST[94]
		CelebA [50]
		Youtube Faces [91]
		GTSRB [80]
		LISA Traffic Sign [61]
		UnityEyes [92]
		Oxford Flowers [66]
	Object Detection	ILSVRC [74]
		Coco [48]
	Driving Video Frames	Driving [82]
	Human Behavior	UCF101 [79]
Fairness Benchmark	Fairness Classification	Berkeley [11]
		Adult [2]
		Statlog (German Credit Data) [1]
		Compas ¹
		Berkeley Admissions [10]
		Titanic ²
		Bank Marketing [62]
		Bank Execution ³
	Recommendation System	Fraud Detection ⁴
		Mep ⁵
		Raw Car Rentals
		MovieLens [31]
	Face Recognition	IJB-A [40]
		Adience [44]
		Pilot Parliaments Benchmark [S50]
	Medical Classification	Heart Disease [3]
		Predictive Healthcare
Natural Language Processing	Text Recognition	Snips [19]
	Text Classification	AgNews [99]
		MoleculeNet [93]
Automatic Speech Recognition	Speech Classification	IMDB reviews [54]
		AN4 [4]
		TIMIT [83]
		TED-LIUM 2 [73]
		TED-LIUM 3 [32]
Healthcare	Medical Classification	Breast Cancer Wisconsin (Diagnostic) [56]
		Depresjon [24]
		Psykose [35]
General Purpose	Malware Recognition	Drebin[7]
	Anomaly Detection	Contagio/Virustotal
		4SICS Geek Lounge ⁶
		BTS prediction ⁷

¹ Compas: <https://github.com/propublica/compas-analysis>
² Titanic: <https://www.kaggle.com/c/titanic/data>
³ Bank Execution: <https://data.world/markmarkoh/executions-since-1977>
⁴ Fraud Detection: <https://www.kaggle.com/c/fraudddetection/data&MarketingCampaigns>
⁵ Mep: https://meps.ahrq.gov/mepsweb/data_stats/download_data_files_detail.jsp?cboPufNumber=HC-181
⁶ 4SICS Geek Lounge: <https://www.netresec.com/?page=PCAP4SICS>
⁷ BTS prediction: <https://github.com/rdsea/loTCloudSamples/tree/master/MLUnits/>

Table 7. Comparison of Machine Learning Models across Various Domains and Tasks.

Domain	Task	Model Architecture
Computer Vision	Image Classification CNN	Inception v3, LeNet-1, LeNet-4, LeNet-5, AlexNet, AllConvNet, WideResNet, DenseNet, DenseNet121, ResNet-18, ResNet-20, ResNet-32, ResNet-38, ResNet-50, ResNet-56, VGG-11, VGG-16, VGG-19, VGGNet, Emotion, SqueezeNet, MobileNet V1, MobileNet V2, GoogleNet, NASNetMobile, NIN, RegNetX, DPN92, ShuffleNet, Dave-orig, Dave-norminit, Dave-dropout
	Image Classification RNN	LSTM, ReNet, GRU
	Object Detection CNN	YOLOv4, COCO SDD, MobileNet V1, Google Mobile Object Localizer, Mask R-CNN
	Image Segmentation CNN	DeepLabv3
	Pose Estimation CNN	PoseNet
	Human Behavior RNN	VGG16+LSTM
Fairness Benchmark	Fairness Classification	K-Means, Logistic Regression, Decision Tree, Random Forest, XGBoost, Support Vector Machine, K-Nearest Neighbors, Discriminant Analysis, Naive Model, Sample Clustering, AdaBoost, ResNet-18, Multi-Layer Perceptron
Natural Language Processing	Sentiment Analysis RNN	LSTM
	Text Regression RNN	LSTM
	Text Classification RNN	Transformer-xl, GNMT
Automatic Speech Recognition	Speech Classification RNN	DeepSpeech, Bi-LSTM, Bi-LSTM, RNN-RNN, Transformer-AED, Conformer-AED
Healthcare	Health Classification	Privacy-Preserving Distributed Extremely Randomized Trees, ID3, ERT, Random Forest, XG-Boost, Decision Tree, Linear SVM
General Purpose	Recommend Items	Alternating Least Squares, NCF
	Vulnerability Prediction RNN	Bi-GRU, Bi-LSTM

the full set of datasets available in the literature; hence we cannot provide conclusive remarks on the unavailability of datasets; as such, we limit ourselves to highlight a potential limitation that should be further investigated.

As a second point of discussion, we could observe that for most application domains multiple tasks were supported. The only exceptions were represented by ‘*Automated Speech Recognition*’ and ‘*Healthcare*’, for which all the datasets available allow to perform speech and medical classification, respectively. The information coming from this analysis may be beneficial for researchers interested in selecting the most appropriate dataset(s) to use within their own research.

Finally, a few datasets were categorized under ‘*General Purpose*’. The tasks enabled by those datasets are limited to reliability analysis and, depending on the specific use made by the automated approaches, may support the identification of quality or security issues.

In conclusion of this thematic analysis, we could report that a variety of tasks may be enabled by the availability of datasets, yet we still pointed out the multiple application domains were not

targeted by the automated approaches proposed in literature so far. As such, our study highlights the need for further domain-specific investigations.

When considering Table 7, we could first confirm that some specific tasks deserved more attention from the research community. Application domains such as ‘*Computer Vision*’ and ‘*Fairness Benchmark*’ are indeed confirmed to be the most investigated ones. Furthermore, researchers in these fields have experimented with multiple alternative solutions, assessing the suitability of several algorithms to deal with the tasks of ‘*Image Classification*’ and ‘*Fairness Classification*’. Interestingly, as shown in Table 7, the ‘*Computer Vision*’ domain has been investigated through various solutions based on both convolutional and recurrent neural networks, while the other domains did not broader investigations. The pieces of information provided by our study could inform researchers on the current state of the artificial intelligence algorithms applied to deal with non-functional requirements of machine learning-enabled systems, possibly informing them of additional perspectives to take into account.

By combining the results coming from our analyses, we could also provide some insights into the nature of the datasets available and the capabilities of the current automated approaches. As we learned from Table 7, most of the developed solutions were based on deep learning algorithms fed through the large datasets shown in Table 6. On the one hand, this information highlights that the datasets available were designed to be large enough to train both shallow and deep learning solutions able to deal with non-functional issues of machine learning-enabled systems: in this respect, our findings report that, despite the intrinsic limitations of the available datasets in terms of domains considered, these might be useful to experiment with multiple artificial intelligence solutions, hence representing potentially relevant benchmarks for the research community. On the other hand, deep learning solutions seem to be the most widely employed in most application domains, possibly indicating that the management of non-functional requirements of machine learning-enabled systems requires advanced learning techniques to properly support practitioners: along this side, further experimentation and comparisons between traditional and deep learning techniques might be worth to be conducted.

4.4.2 On the automated approaches and their relevance. Table 8 lists the specific approaches proposed so far to deal with non-functional requirements of machine learning-enabled systems. The table classifies each approach based on multiple characteristics such as (1) the development phase it can be applied, i.e., we classified them based on whether they can be applied on the phases of ‘*Data Engineering*’ (a.k.a. pre-processing), ‘*Training*’ (a.k.a. in-processing), and ‘*Post-training*’ (a.k.a. post-processing); (2) the non-functional requirements targeted, i.e., a ‘☺’ symbol indicates that an approach considered the non-functional requirement, while a ‘✖’ symbol indicates that the approach had not taken a non-functional requirements into account. The table also reports a description of the currently available approaches, hence providing the reader with a quick summary of the capabilities of each of them. Looking at the table, we could first highlight that most of the efforts devoted by the SE4AI community focused on post-processing approaches, with a particular focus on software testing and automated program repair. Pre- and in-processing approaches are present in a lower extent, possibly suggesting that future research might target those aspects to further support practitioners. Secondly, we could also find out that accuracy is the most widely-considered non-functional requirements: as a matter of fact, the vast majority of the automated approaches focused on accuracy as a primary objective to safeguard—this indicates that accuracy is still the most relevant non-functional property to consider while developing automated approaches.

Perhaps more importantly for the scope of this paper, we could understand that there is no approach able to comprehensively analyze the whole set of non-functional requirements classified

in the context of RQ_1 . As a matter of fact, only a few approaches optimized two or more non-functional requirements and, more interestingly, only a few approaches were deemed to be relevant for non-functional aspects such as efficiency, maintainability, and usability, hence suggesting that more research should be done on the optimization of those concerns.

As an additional perspective, we could see that many of the challenges identified in RQ_2 are still not addressed by currently available approaches. In other terms, while the literature called for research in various non-functional requirements domains, there still seems to be a little automated tooling able to address it. For instance, we could observe little to no automated solutions available for solving privacy, post-deployment simulations, legal aspects, and adversarial learning. As such, the final outcome of our literature review points out, in the first place, the lack of automated approaches able to comprehensively consider non-functional requirements of machine learning-enabled systems. Also, our work highlights that most of the challenges identified in the context of RQ_2 are still open and serve as a way to inform our research community on the next steps to perform to provide additional support to practitioners.

🔑 **Answer to RQ_3 .** Our RQ_3 finally points out that the currently available support is limited and, indeed, most of the software engineering challenges identified are still open and worth to further investigation. In addition, the currently available approaches do not often perform a comprehensive optimization of non-functional requirements; as such, practitioners would be required to deploy a complex selection of tools to optimize relevant requirements, not being able to rely on multi-objective approaches that may find trade-offs among desired design aspects.

Table 8. Automated approach to optimize NFRs: Accuracy (A), Efficiency (E), Maintainability (M), Resiliency (R), Sustainability (S), Usability (U).

ML Pipeline	Approach	Description	A	E	M	R	S	U
Data-driven approach	Fairway [S34]	This paper discusses the issue of algorithmic discrimination and ethical bias in machine learning software. It proposes a method called Fairway to detect and mitigate bias in the training data and the learned model. The method combines pre-processing and in-processing approaches and removes ethical bias without significantly affecting predictive accuracy. The paper advocates for testing and mitigating bias to become a routine part of the machine learning software development life cycle	☺	✗	✗	✗	☺	☺
	Deepstate [S20]	The paper proposes DeepState, a test suite selection tool for Recurrent Neural Network (RNN) models to reduce data labeling and computation costs. DeepState selects data based on a stateful perspective of RNN and identifies possibly misclassified tests by capturing neuron state changes. The tool further provides a test selection method to obtain a test suite with strong fault detection and model improvement capabilities	☺	✗	✗	☺	☺	✗
	Dare [S12]	The paper proposes a novel model training framework called Dare that aims to improve the universal robustness of deep learning (DL) models. The proposed approach incorporates model transformation and data augmentation in a delta debugging fashion	☺	✗	✗	☺	✗	✗
	MAAT [S13]	This paper proposes a novel ensemble approach, MAAT, to improve the fairness-performance trade-off for machine learning (ML) software. MAAT combines models optimized for different objectives: fairness and ML accuracy.	☺	✗	✗	✗	☺	✗
Training approach	Parfait-ML [S21]	This paper investigates the impact of hyperparameters on fairness in machine learning (ML) algorithms. The authors propose three search-based software testing algorithms to identify hyperparameters that can improve fairness without sacrificing precision. They use statistical debugging to explain the role of these parameters in improving fairness and implement the proposed approach in the tool Parfait-ML.	☺	✗	✗	✗	☺	☺
Continued on next page								

Table 8 – continued from previous page

ML Pipeline	Approach	Description	A	E	M	R	S	U
Training approach	Decomposing Convolutional Neural Networks into Reusable and Replaceable Modules [S25]	The paper proposes a method to decompose a Convolutional Neural Network (CNN) model for image classification into modules for each output class, enabling reusability and replaceability. Reusing and replacing these modules reduces CO2e emissions	⊙	⊙	⊙	✗	⊙	✗
	k-PPD-ERT [S36]	This paper proposes a distributed extremely randomized trees (ERT) algorithm for machine learning on healthcare data without compromising patients’ privacy. The proposed method, k-PPD-ERT, uses a cloud platform and is tested on medical datasets, including Heart Disease, Breast Cancer, and two mental health datasets.	⊙	✗	✗	⊙	⊙	✗
	DeepCorrect [S10]	This paper discusses the impact of image distortions, such as blur and noise, on pre-trained convolutional filters in deep neural networks used for computer vision tasks. They introduced a new approach called DeepCorrect. It applies small stacks of convolutional layers with residual connections to correct the worst affected filter activations without changing the rest of the pre-trained filter outputs. The performance results demonstrate that DeepCorrect significantly improves the robustness of DNNs against distorted images	⊙	✗	✗	⊙	✗	✗
	Distributed Selective SGD [S8]	The paper proposes a system that allows multiple parties to learn an accurate neural-network model without sharing their datasets, by selectively sharing small subsets of their models’ key parameters during training. This system preserves privacy while still improving learning accuracy beyond what is achievable on their inputs	⊙	✗	✗	⊙	✗	✗
	Towards Training Reproducible Deep Learning Models [S16]	The paper proposes a systematic approach to training reproducible Deep Learning (DL) models, which is crucial for various tasks like training, testing, debugging, and auditing. DL models are challenging to reproduce due to issues like randomness in the software and non-determinism in the hardware. The proposed approach includes general criteria to evaluate reproducibility, a framework using record-and-replay and profile-and-patch techniques to mitigate software-related randomness and hardware-related non-determinism, and a reproducibility guideline	⊙	✗	⊙	✗	✗	✗
Post-training approach	NNREPAIR [S2]	NNREPAIR tool use a technique for repairing neural network classifiers by fixing potentially faulty network parameters. It uses fault localization to identify the faulty parameters and then performs repairs using constraint solving to make small parameter modifications. The technique can improve the accuracy of models, fix security vulnerabilities caused by the poisoning of training data, and improve the robustness of the network against adversarial attacks.	⊙	✗	✗	⊙	✗	✗
	AI-Lancet [S9]	AI-Lancet proposes a systematic approach to trace and fixes errors in deep learning models that could lead to severe security issues. They locate error-inducing neurons and propose two methods, neuron-flip, and neuron-fine-tuning, to fix the errors.	⊙	✗	✗	⊙	✗	⊙
	Apricot [S11]	The paper presents Apricot, a novel weight-adaptation approach to fixing imprecise deep learning models. Apricot generates a set of reduced DL models from the original DL model. In each iteration, it adjusts the weights of the input DL model towards the average weight of the reduced DL models that correctly classify a failing test case and/or away from that of the reduced DL models that misclassify the same test case.	⊙	✗	✗	⊙	✗	✗
	FairNeuron [S17]	The paper proposes a tool called FairNeuron, which automatically repairs Deep Neural Network (DNN) models to balance accuracy and fairness trade-offs by detecting neurons with contradictory optimization directions from accuracy and fairness training goals and achieving a trade-off by selective dropout. The approach is lightweight, scales to large models, and is more efficient than state-of-the-art methods.	⊙	✗	✗	⊙	⊙	⊙
	RobOT [S23]	This paper proposes a testing framework called Robustness-Oriented Testing (RobOT) for deep learning (DL) systems that aims to improve model robustness. RobOT utilizes a novel quantitative measurement that evaluates the value of each test case in improving model robustness and the convergence quality of the model robustness improvement.	⊙	✗	⊙	⊙	✗	✗
	Search-based test and improvement of machine-learning-based anomaly detection systems [S47]	This paper focuses on testing intrusion detection systems (IDS) against training attacks, which are designed to exploit the vulnerability of machine-learning-based anomaly detection systems. The authors propose a search-based approach to test IDS by making training attacks, and they also propose searching for countermeasures to increase the resilience of the tested IDS.	✗	✗	⊙	⊙	✗	✗
Continued on next page								

Table 8 – continued from previous page

ML Pipeline	Approach	Description	A	E	M	R	S	U
	ASRTTest [S28]	The paper proposes a testing approach for deep neural network-driven automatic speech recognition (ASR) systems called ASRTTest, which is built on the theory of metamorphic testing. The approach involves designing metamorphic relations for ASR systems and implementing transformation operators to simulate practical scenarios that generate speeches. Gini impurity is used to guide the generation process and improve testing efficiency.	⊙	⊙	×	⊙	×	×
	Hybrid Repair [S29]	The paper proposes HybridRepair, an annotation-efficient solution for repairing deep learning models with a mismatch between the distribution of training data and field data. The solution takes a holistic approach that combines a small amount of annotated data with a large amount of unlabeled data, leveraging semi-supervised learning techniques to boost training data density. The proposed method selectively annotates data in failure regions and propagates labels to neighboring data while using a selected part of the unlabeled data to avoid the impact of distribution shift on SSL solutions.	⊙	×	×	⊙	×	×
Post-training approach	BET [S30]	The paper proposes a Black-box Efficient Testing (BET) method for identifying defects in convolutional neural networks (CNNs) without requiring full knowledge of the target model. The BET method generates continuous perturbations in a black-box manner and uses a tunable objective function to guide the testing process. The method also employs an efficiency-centric policy to find error-inducing inputs within a fixed query budget	⊙	×	×	⊙	×	×
	Security for distributed deep neural networks: Towards data confidentiality & intellectual property protection [S32]	This paper proposes an approach for protecting distributed deep neural network (DNN) based software assets, including the confidentiality of input and output data streams and safeguarding intellectual property, using fully homomorphic encryption (FHE). The authors evaluate the feasibility of this approach on a convolutional neural network (CNN) for image classification deployed.	⊙	⊙	×	⊙	×	×
	Interpreting Deep Learning-based Vulnerability Detector Predictions Based on Heuristic Searching [S38]	The article proposes a new explanation framework to identify the small number of tokens that contribute significantly to the detector's prediction, which can help domain experts understand and accept the detector's outputs. The experiments show that this framework has higher fidelity than existing methods, especially in real-world scenarios where features are not independent.	×	×	×	⊙	×	⊙
	LEMNA [S39]	The paper proposes a high-fidelity explanation method called LEMNA for security applications, which generates a small set of interpretable features to explain how the input sample is classified. LEMNA approximates a local area of the deep learning decision boundary using a simple interpretable model that handles feature dependency and nonlinear local boundaries to improve explanation fidelity.	×	×	×	⊙	⊙	⊙

5 DISCUSSION AND IMPLICATIONS

The results coming from the our study provide a number of additional discussion points and implications that we further discuss in the following.

On the Relation between the acquired knowledge and the previous one. Our results allowed us to discover multiple aspects concerned with non-functional requirements of machine learning-enabled systems that extend the existing knowledge. In the first place, the outcome of **RQ₁** provided a comprehensive classification of non-functional requirements along with their classes. In this respect, our work extended the insights provided by Habibullah et al. [28], providing a larger overview of the issues that practitioners should consider while developing machine learning-enabled systems. Such an additional knowledge may serve as an input to both researchers and practitioners. The former might take our review as a means to design empirical experimentation and approaches able to deal with the non-functional aspects that previous research did not consider. The former might instead take our classification as an input to increase their own awareness with respect to the problems arising when developing machine learning-enabled systems, other than reason on how to address those aspects in real-world use cases.

› **Take Away Message.** Our classification of non-functional requirements may inspire further research on the matter, other than making practitioners aware of the multiple, multi-faceted concerns arising when developing machine learning-enabled software systems.

At the same time, RQ_1 may possibly support the definition of standards and guidelines that attempt to appropriately describe how to deal with non-functional requirements of machine learning-enabled systems. For example, the definitions provided might be potentially useful to support and complement an existing, emerging standard such as the *ISO/IEC 25059*,¹² which defines an initial taxonomy of non-functional properties relevant for machine learning-enabled systems. In this respect, we foresee improved implementations of the standard, which takes our findings into account, hence enlarging the definitions available for researchers and practitioners.

› **Take Away Message.** The classification exercise made in the context of RQ_1 may be useful to extend and/or complement emerging standards describing relevant non-functional requirements of machine learning-enabled systems.

As a final point of this discussion, it is also worth commenting on how the findings coming from RQ_2 and RQ_3 inform future research. Our research identified a set of over 20 software engineering challenges that should be further considered by our community. When comparing our findings with those reported by Horkoff [33], we could corroborate some of the challenges identified, like the need for engineering machine learning-enabled systems for reducing internal errors and security, yet we could elaborate a larger, comprehensive catalog of challenges that have to do with multiple aspects of SE4AI and that naturally inform a number of software engineering research communities, from the one on requirements engineering till the one on software testing. As such, our work can be considered as a cross-cutting knowledge base for serving further research in software engineering. The outcomes of RQ_3 reinforce our conclusion: as many non-functional aspects were not still considered or partially taken into account by existing automated approaches, additional research would be worthy. Our overview of both challenges and automated approaches provides a ground for the development of additional software engineering instruments.

› **Take Away Message.** Our work could identify a number of software engineering challenges that are not yet targeted by automated approaches, hence suggesting the future avenues that researchers can consider to further enlarge the knowledge and support provided to handle non-functional requirements of machine learning-enabled systems.

On the Inter-Relation among Non-Functional Requirements. According to our results, and specifically, those coming from RQ_2 , there exist some inter-relations among non-functional requirements. There are not only concerned with the relation that accuracy has with other requirements but also with the innate interconnections between multiple non-functional aspects playing a role in the development of machine learning-enabled systems. The set of challenges identified in our work could provide insights into the multi-faceted engineering of non-functional requirements, hence providing researchers with information about the challenges to simultaneously face when devising automated approaches. At the same time, the results of RQ_3 further inform our research community, other than practitioners. In the first place, we could point out that the available automated approaches could only satisfy sub-sets of non-functional requirements, hence not being comprehensive enough to properly support a holistic development of machine learning-enabled systems. At the same time, practitioners could be informed of the support available, possibly highlighting opportunities for technological transfer. In any case, our

¹²The *ISO/IEC 25059* standard: <https://iso25000.com/index.php/en/iso-25000-standards/iso-25059>

work represents a call for a multi-objective, comprehensive investigation into the non-functional attributes of machine learning-enabled systems and their interconnections.

› **Take Away Message.** Our findings suggest novel interconnections and research on the multi-objective optimization of non-functional requirements of machine learning-enabled systems. We call for a brand new research field focusing on empirically investigating the relations between non-functional requirements and how those relations may inform the development of automated approaches to optimize machine learning-enabled systems.

A Managerial Viewpoint. As a follow-up discussion, the research questions targeted by our work let arise multiple trade-offs to consider when developing machine learning-enabled systems. As a matter of fact, the accuracy of artificial intelligence algorithms must necessarily be balanced with other aspects to create trustworthy software systems. On the one hand, this clearly represents a call for further research. On the other hand, our work may have implications on the managerial side: the constant and continuous search for trade-offs indeed represents a high-level challenges for project managers, who are required to monitor and handle multiple non-functional attributes throughout the evolution of machine learning-enabled systems. In the first place, the results produced by **RQ₁** and **RQ₃** may allow the reader to understand what is the set of non-functional attributes to actually take into account and the approaches currently available to optimize them, respectively. Secondly, the challenges identified in the context of **RQ₂** should not only be considered from a technical perspective but also from a socio-technical and managerial one. Our work indeed calls for further research on the managerial strategies that would indicate the most appropriate management policies to apply when handling non-functional requirements. Also, we bring to the attention of software engineering and software project management researchers the lack of optimization approaches that may be contextual, dependent, and adaptable to the evolution of the system—we argue that those properties would be key to enabling the proper management of non-functional requirements over the software lifecycle.

› **Take Away Message.** We call for research on managerial and socio-technical strategies to handle non-functional requirements throughout the evolution of machine learning-enabled systems. At the same time, our findings point out the need for optimization approaches specifically tailored to software evolution, hence being contextual, dependent, and adaptable.

Machine Learning Sustainability: Software Engineering Comes to Rescue. The results of both **RQ₂** and **RQ₃** identified sustainability as a non-functional requirement that has been somehow neglected by our research community. While previous work has started exploring the social side of sustainability, we could identify a lack of software engineering investigations into energy- and cost-related concerns. On the one hand, our work identified *tiny machine learning* [47] as a promising solution to some of the key challenges affecting the energy consumption of machine learning-enabled systems: as such, we call for additional investigations into these aspects. On the other hand, we also highlight a lack of experimentation in terms of costs: in this respect, further analysis of how to predict and manage costs, other than how to optimize non-functional approaches by taking those aspects into consideration, would be worthwhile.

› **Take Away Message.** Sustainability-related concerns should be more carefully considered by the software engineering research community. There is indeed a need for further approaches to deal with energy consumption other than cost-aware non-functional requirements optimization strategies.

On the Currently Available Automated Support. Both RQ_2 and RQ_3 pointed out the limited amount of automated approaches, other than their low generalizability. In addition, we could also realize the limited support provided by researchers in terms of the analysis of specific contexts, architectures, and algorithms. Hence, it is our hope that the findings reported may represent a call for researchers working in the field of automated software engineering to propose novel techniques and solutions able to support practitioners in a broader manner and in a way that faces the specific challenges pertaining to the management of non-functional requirements.

› **Take Away Message.** Our findings clearly point out the limited automated support that practitioners have, other than the limited generalizability of the available solutions. Our work represents a call for automated software engineering research efforts aiming at facing the specific challenges identified and providing novel, generalizable approaches to handle and optimize non-functional requirements of machine learning systems in practice.

6 THREATS TO VALIDITY

The validity of both the research method and conclusions drawn in our systematic literature review might have been threatened by some key considerations. This section overviews the potential limitations of our study and how these were mitigated when designing the study.

Literature selection. A critical challenge for a systematic literature review is identifying a consistent and comprehensive body of knowledge concerning the subject of interest. In this respect, we first approached the search process by adhering to well-established guidelines [38]. Nonetheless, we realized that these guidelines would not have been sufficient to extract the required body of knowledge and, as a consequence, might have led to missing key resources. To mitigate this threat, we, therefore, opted for a *hybrid* systematic analysis [63]: we complemented the set of primary studies identified through the guidelines by Kitchenham et al. [38] with additional resources coming from (1) the systematic screening of the research articles published in top-tier software engineering venues [90]; and (2) multiple snowballing iterations of the incoming and outgoing references of the primary studies identified [89].

In the second place, it is worth remarking that we performed the search on multiple databases such as *ACM Digital Library*, *Scopus*, and *IEEE Xplore*. This was done to ensure wider coverage of the primary studies published in the literature.

The reliance on a hybrid systematic literature review, other than the multiple actions conducted to extend the search process, makes us confident of the completeness of the literature selection. However, for the sake of verifiability and replicability, our online appendix [58] contains all the data and material used to produce each intermediate search of our study. The interested reader might use the material to either assess the soundness/completeness of the process and further build on top of our results [90].

Literature analysis and synthesis. Upon completing the search process, we applied several steps to ensure the inclusion of the relevant primary studies to address the research questions driving our systematic literature review. In the first place, we designed a set of exclusion criteria with the intent of filtering out the primary studies that did not fit our scope. In addition, we proceeded with the definition of inclusion criteria, other than a formal quality assessment of the suitability of each identified resource for the goals of our study.

These steps were conducted manually; therefore, the risk of subjectiveness and human error were the main limitations. While the first author mainly conducted all the activities, there are two considerations to make. In the first place, we designed the systematic literature review so that validation was conducted at the end of each step. The validation first involved the second author

of the paper, who took the role of inspector: he was indeed called to verify the actions conducted by the first author and identify potential errors. At the end of this process, both authors opened a discussion and addressed the concerns raised in the validation.

Secondly, the two authors constantly worked together to define the research method and actions to make the study sound and reliable. In particular, they met weekly to discuss the advances of the systematic literature review and the potential limitations to address. These weekly meetings further limited potential concerns due to subjectiveness and human bias. Nonetheless, we acknowledge that the entire process has been conducted based on the knowledge and expertise of the authors, which might be limited. On the one hand, we released all the material produced to be as transparent as possible [58]. On the other hand, we can only report some background information - this may be useful to assess the study's validity and estimate the resources required to replicate our work. The two authors have a research experience of one and ten years, respectively. Both conduct or have already conducted quantitative and qualitative studies in the past, other than systematic literature/mapping studies on themes connected to software engineering for artificial intelligence, optimization of non-functional requirements, and software maintenance and evolution. Furthermore, they are both involved in the academic courses of *Software Engineering*, *Fundamentals of Artificial Intelligence*, and *Software Engineering for Artificial Intelligence* at the University of Salerno (Italy) - the second author is the lecturer of those courses, while the first is a teaching assistant.

7 CONCLUSION

In this paper, we conducted a hybrid systematic literature review on non-functional requirements of machine learning-enabled systems—this was motivated by the increasing number of papers published on these aspects during the last ten years. We tackled three main research angles such as (1) the classification of the non-functional requirements investigated so far, (2) the challenges to face when dealing with them, and (3) the automated approaches proposed in literature to support practitioners when optimizing them in practice. The results of the systematic literature review do not only summarize the current knowledge on the matter, but also opened new horizons and challenges that our research community should consider. It is our hope that researchers and fresh Ph.D. Students might be inspired by our work and, further contributing to the field. All in all, our work brings the following major contributions:

- (1) A systematic literature review on non-functional attributes of machine learning-intensive systems, which allowed to systematically classify them, other than identify the current open challenges and approaches to cope with them. These pieces of information may be exploited by researchers to define the next research steps to improve the currently available techniques and overall support provided to practitioners;
- (2) A set of implications and take-away messages that researchers may use to address future research avenues on the management and optimization of non-functional requirements of machine learning-enabled systems;
- (3) An online appendix containing all data and scripts used in the study, which might be used to extend the scope of the study or replicate our work.

The considerations and implications of this systematic literature review drive our future research agenda. We will indeed work toward the definition of novel approaches that could optimize the non-functional requirements of machine learning-enabled systems while keeping accuracy into account. In addition, we aim to develop empirical investigations to understand the impact of non-functional requirements other than the practitioners' expectations with respect to that.

CREDITS

Vincenzo De Martino: Formal analysis, Investigation, Data Curation, Validation, Writing - Original Draft, Visualization. **Fabio Palomba:** Supervision, Validation, Writing - Review & Editing.

CONFLICT OF INTEREST

The authors declare that they have no conflict of interest.

DATA AVAILABILITY

The data collected in the context of this systematic literature review, along with the scripts used to analyze and generate data, charts, and plots discussed when addressing our research goals, are publicly available at: [58]

ACKNOWLEDGEMENT

Fabio is partially funded by the Swiss National Science Foundation through the SNF Projects No. PZ00P2_186090. This work has been partially supported by the EMELIOT national research project, which has been funded by the MUR under the PRIN 2020 program (Contract 2020W3A5FY).

REFERENCES

- [1] 1994. UCI Statlog (German Credit Data) Data Set. [https://archive.ics.uci.edu/ml/datasets/Statlog+\(German+Credit+Data\)](https://archive.ics.uci.edu/ml/datasets/Statlog+(German+Credit+Data))
- [2] 1996. UCI Adult Data Set. <https://archive.ics.uci.edu/ml/datasets/adult>
- [3] 2001. UCI heart disease data set. <https://archive.ics.uci.edu/ml/datasets/Heart+Disease>
- [4] Alex Acero. 1992. *Acoustical and environmental robustness in automatic speech recognition*. Vol. 201. Springer Science & Business Media.
- [5] Khlood Ahmad, Mohamed Abdelrazek, Chetan Arora, Muneera Bano, and John Grundy. 2023. Requirements engineering for artificial intelligence systems: A systematic mapping study. *Information and Software Technology* (2023), 107176.
- [6] Asad Ali and Carmine Gravino. 2019. A systematic literature review of software effort prediction using machine learning methods. *Journal of Software: Evolution and Process* 31, 10 (2019), e2211. <https://doi.org/10.1002/smr.2211> arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1002/smr.2211> e2211 JSME-18-0247.R2.
- [7] Daniel Arp, Michael Spreitzenbarth, Malte Hubner, Hugo Gascon, Konrad Rieck, and CERT Siemens. 2014. Drebin: Effective and explainable detection of android malware in your pocket.. In *Ndss*, Vol. 14. 23–26.
- [8] Muhammad Ilyas Azeem, Fabio Palomba, Lin Shi, and Qing Wang. 2019. Machine learning techniques for code smell detection: A systematic literature review and meta-analysis. *Information and Software Technology* 108 (2019), 115–138.
- [9] Osbert Bastani, Yani Ioannou, Leonidas Lampropoulos, Dimitrios Vytiniotis, Aditya Nori, and Antonio Criminisi. 2016. Measuring Neural Net Robustness with Constraints. In *Advances in Neural Information Processing Systems*, D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett (Eds.), Vol. 29. Curran Associates, Inc. <https://proceedings.neurips.cc/paper/2016/file/980ecd059122ce2e50136bda65c25e07-Paper.pdf>
- [10] P. J. Bickel, E. A. Hammel, and J. W. O'Connell. 1975. Sex Bias in Graduate Admissions: Data from Berkeley. *Science* 187, 4175 (1975), 398–404. <https://doi.org/10.1126/science.187.4175.398> arXiv:<https://www.science.org/doi/pdf/10.1126/science.187.4175.398>
- [11] Peter J Bickel, Eugene A Hammel, and J William O'Connell. 1975. Sex Bias in Graduate Admissions: Data from Berkeley: Measuring bias is harder than is usually assumed, and the evidence is sometimes contrary to expectation. *Science* 187, 4175 (1975), 398–404.
- [12] Manal Binkhonain and Liping Zhao. 2019. A review of machine learning algorithms for identification and classification of non-functional requirements. *Expert Systems with Applications: X* 1 (2019), 100001.
- [13] Markus Borg, Cristofer Englund, Krzysztof Wnuk, Boris Duran, Christoffer Levandowski, Shenjian Gao, Yanwen Tan, Henrik Kaijser, Henrik Lönn, and Jonas Törnqvist. 2018. Safely entering the deep: A review of verification and validation for machine learning and a challenge elicitation in the automotive industry. *arXiv preprint arXiv:1812.05389* (2018).
- [14] Pierre Bourque and E Richard. 2014. Swebok Version 3.0. *IEEE, ISBN-10: 0-7695-5166-1* (2014).
- [15] Housseem Ben Braiek and Foutse Khomh. 2020. On testing machine learning programs. *Journal of Systems and Software* 164 (2020), 110542.
- [16] Yuriy Brun and Alexandra Meliou. 2018. Software fairness. In *Proceedings of the 2018 26th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 754–759.

- [17] Victor R Basili, Gianluigi Caldiera, and H Dieter Rombach. 1994. The goal question metric approach. *Encyclopedia of software engineering* (1994), 528–532.
- [18] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [19] Alice Coucke, Alaa Saade, Adrien Ball, Théodore Bluche, Alexandre Caulier, David Leroy, Clément Doumouro, Thibault Gisselbrecht, Francesco Caltagirone, Thibaut Lavril, et al. 2018. Snips voice platform: an embedded spoken language understanding system for private-by-design voice interfaces. *arXiv preprint arXiv:1805.10190* (2018).
- [20] Elder Vicente de Paulo Sobrinho, Andrea De Lucia, and Marcelo de Almeida Maia. 2018. A systematic literature review on bad smells–5 w’s: which, when, what, who, where. *IEEE Transactions on Software Engineering* 47, 1 (2018), 17–66.
- [21] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*. 248–255. <https://doi.org/10.1109/CVPR.2009.5206848>
- [22] Simone Disabato and Manuel Roveri. 2022. Tiny machine learning for concept drift. *IEEE Transactions on Neural Networks and Learning Systems* (2022).
- [23] Sainyam Ghalotra, Yuriy Brun, and Alexandra Meliou. 2017. Fairness testing: testing software for discrimination. In *Proceedings of the 2017 11th Joint meeting on foundations of software engineering*. 498–510.
- [24] Enrique Garcia-Ceja, Michael Riegler, Petter Jakobsen, Jim Tørresen, Tine Nordgreen, Ketil J. Oedegaard, and Ole Bernt Fasmer. 2018. Depresjon: A Motor Activity Database of Depression Episodes in Unipolar and Bipolar Patients. In *Proceedings of the 9th ACM Multimedia Systems Conference (Amsterdam, Netherlands) (MMSys ’18)*. Association for Computing Machinery, New York, NY, USA, 472–477. <https://doi.org/10.1145/3204949.3208125>
- [25] Bahar Gezici and Ayça Kolukisa Tarhan. 2022. Systematic literature review on software quality for AI-based software. *Empirical Software Engineering* 27, 3 (2022), 66.
- [26] Gökem Giray. 2021. A software engineering perspective on engineering machine learning systems: State of the art and challenges. *Journal of Systems and Software* 180 (2021), 111031.
- [27] Tianyu Gu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2019. BadNets: Evaluating Backdooring Attacks on Deep Neural Networks. *IEEE Access* 7 (2019), 47230–47244. <https://doi.org/10.1109/ACCESS.2019.2909068>
- [28] Khan Mohammad Habibullah, Gregory Gay, and Jennifer Horkoff. 2022. Non-functional requirements for machine learning: An exploration of system scope and interest. In *2022 IEEE/ACM 1st International Workshop on Software Engineering for Responsible Artificial Intelligence (SE4RAI)*. IEEE, 29–36.
- [29] Khan Mohammad Habibullah and Jennifer Horkoff. 2021. Non-functional requirements for machine learning: understanding current use and challenges in industry. In *2021 IEEE 29th International Requirements Engineering Conference (RE)*. IEEE, 13–23.
- [30] Gaétan Hains, Arvid Jakobsson, and Youry Khmelevsky. 2018. Towards formal methods and software engineering for deep learning: security, safety and productivity for dl systems development. In *2018 Annual IEEE international systems conference (syscon)*. IEEE, 1–5.
- [31] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Trans. Interact. Intell. Syst.* 5, 4, Article 19 (dec 2015), 19 pages. <https://doi.org/10.1145/2827872>
- [32] François Hernandez, Vincent Nguyen, Sahar Ghannay, Natalia Tomashenko, and Yannick Estève. 2018. TED-LIUM 3: Twice as Much Data and Corpus Repartition for Experiments on Speaker Adaptation. In *Speech and Computer*, Alexey Karpov, Oliver Jokisch, and Rodmonga Potapova (Eds.). Springer International Publishing, Cham, 198–208.
- [33] Jennifer Horkoff. 2019. Non-functional requirements for machine learning: Challenges and new directions. In *2019 IEEE 27th International Requirements Engineering Conference (RE)*. IEEE, 386–391.
- [34] Ling Huang, Anthony D. Joseph, Blaine Nelson, Benjamin I.P. Rubinstein, and J. D. Tygar. 2011. Adversarial Machine Learning. In *Proceedings of the 4th ACM Workshop on Security and Artificial Intelligence* (Chicago, Illinois, USA) (*AISeC ’11*). Association for Computing Machinery, New York, NY, USA, 43–58. <https://doi.org/10.1145/2046684.2046692>
- [35] Petter Jakobsen, Enrique Garcia-Ceja, Lena Antonsen Stabell, Ketil Joachim Oedegaard, Jan Oystein Berle, Vajira Thambawita, Steven Alexander Hicks, Pål Halvorsen, Ole Bernt Fasmer, and Michael Alexander Riegler. 2020. PSYKOSE: A Motor Activity Database of Patients with Schizophrenia. In *2020 IEEE 33rd International Symposium on Computer-Based Medical Systems (CBMS)*. 303–308. <https://doi.org/10.1109/CBMS49503.2020.00064>
- [36] Meenu Mary John, Helena Holmström Olsson, and Jan Bosch. 2021. Architecting AI deployment: a systematic review of state-of-the-art and state-of-practice literature. In *Software Business: 11th International Conference, ICSOB 2020, Karlskrona, Sweden, November 16–18, 2020, Proceedings 11*. Springer, 14–29.
- [37] Aaiza Khan, Isma Farah Siddiqui, Mehwish Shaikh, Shabana Anwar, and Murk Shaikh. 2022. Handling non-functional requirements in IoT-based machine learning systems. In *2022 Joint International Conference on Digital Arts, Media and Technology with ECTI Northern Section Conference on Electrical, Electronics, Computer and Telecommunications Engineering (ECTI DAMT & NCON)*. IEEE, 477–479.

- [38] Barbara Kitchenham, O Pearl Brereton, David Budgen, Mark Turner, John Bailey, and Stephen Linkman. 2009. Systematic literature reviews in software engineering—a systematic literature review. *Information and software technology* 51, 1 (2009), 7–15.
- [39] Barbara Kitchenham and Stuart Charters. 2007. Guidelines for performing Systematic Literature Reviews in Software Engineering. 2 (01 2007).
- [40] Brendan F Klare, Ben Klein, Emma Taborsky, Austin Blanton, Jordan Cheney, Kristen Allen, Patrick Grother, Alan Mah, and Anil K Jain. 2015. Pushing the frontiers of unconstrained face detection and recognition: Iarpa janus benchmark a. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 1931–1939.
- [41] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images. (2009).
- [42] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2017. ImageNet Classification with Deep Convolutional Neural Networks. *Commun. ACM* 60, 6 (may 2017), 84–90. <https://doi.org/10.1145/3065386>
- [43] Fumihiro Kumeno. 2019. Software engineering challenges for machine learning applications: A literature review. *Intelligent Decision Technologies* 13, 4 (2019), 463–476.
- [44] Sebastian Lapuschkin, Alexander Binder, Klaus-Robert Muller, and Wojciech Samek. 2017. Understanding and comparing deep neural networks for age and gender classification. In *Proceedings of the IEEE international conference on computer vision workshops*. 1629–1638.
- [45] Yann LeCun. 1998. The MNIST database of handwritten digits. <http://yann.lecun.com/exdb/mnist/> (1998).
- [46] Guofu Li, Pengjia Zhu, Jin Li, Zheming Yang, Ning Cao, and Zhiyi Chen. 2018. Security matters: A survey on adversarial machine learning. *arXiv preprint arXiv:1810.07339* (2018).
- [47] Ji Lin, Wei-Ming Chen, Yujun Lin, Chuang Gan, Song Han, et al. 2020. Mccnet: Tiny deep learning on iot devices. *Advances in Neural Information Processing Systems* 33 (2020), 11711–11722.
- [48] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. 2014. Microsoft coco: Common objects in context. In *Computer Vision—ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part V* 13. Springer, 740–755.
- [49] Bo Liu, Ming Ding, Sina Shaham, Wenny Rahayu, Farhad Farokhi, and Zihuai Lin. 2021. When machine learning meets privacy: A survey and outlook. *ACM Computing Surveys (CSUR)* 54, 2 (2021), 1–36.
- [50] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. 2015. Deep Learning Face Attributes in the Wild. In *Proceedings of International Conference on Computer Vision (ICCV)*.
- [51] Sin Kit Lo, Qinghua Lu, Chen Wang, Hye-Young Paik, and Liming Zhu. 2021. A systematic literature review on federated machine learning: From a software engineering perspective. *ACM Computing Surveys (CSUR)* 54, 5 (2021), 1–39.
- [52] Giuliano Lorenzoni, Paulo Alencar, Nathalia Nascimento, and Donald Cowan. 2021. Machine learning model development from a software engineering perspective: A systematic literature review. *arXiv preprint arXiv:2102.07574* (2021).
- [53] Lucy Ellen Lwakatare, Aiswarya Raj, Ivica Crnkovic, Jan Bosch, and Helena Holmström Olsson. 2020. Large-scale machine learning systems in real-world industrial settings: A review of challenges and solutions. *Information and software technology* 127 (2020), 106368.
- [54] Andrew Maas, Raymond E Daly, Peter T Pham, Dan Huang, Andrew Y Ng, and Christopher Potts. 2011. Learning word vectors for sentiment analysis. In *Proceedings of the 49th annual meeting of the association for computational linguistics: Human language technologies*. 142–150.
- [55] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. 2017. Towards Deep Learning Models Resistant to Adversarial Attacks. <https://doi.org/10.48550/ARXIV.1706.06083>
- [56] Olvi L Mangasarian, W Nick Street, and William H Wolberg. 1995. Breast cancer diagnosis and prognosis via linear programming. *Operations research* 43, 4 (1995), 570–577.
- [57] Silverio Martínez-Fernández, Justus Bogner, Xavier Franch, Marc Oriol, Julien Siebert, Adam Trendowicz, Anna Maria Vollmer, and Stefan Wagner. 2022. Software engineering for AI-based systems: a survey. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 2 (2022), 1–59.
- [58] Vincenzo De Martino and Fabio Palomba. 2023. Classification, Challenges, and Automated Approaches to Handle Non-Functional Requirements in ML-Enabled Systems: A Systematic Literature Review. (5 2023). <https://doi.org/10.6084/m9.figshare.22815770.v1>
- [59] Satoshi Masuda, Kohichi Ono, Toshiaki Yasue, and Nobuhiro Hosokawa. 2018. A survey of software quality for machine learning applications. In *2018 IEEE International conference on software testing, verification and validation workshops (ICSTW)*. IEEE, 279–284.
- [60] Claire Cain Miller. 2015. Can an algorithm hire better than a human. *The New York Times* 25 (2015).
- [61] Andreas Mogelmose, Mohan Manubhai Trivedi, and Thomas B. Moeslund. 2012. Vision-Based Traffic Sign Detection and Analysis for Intelligent Driver Assistance Systems: Perspectives and Survey. *IEEE Transactions on Intelligent Transportation Systems* 13, 4 (2012), 1484–1497. <https://doi.org/10.1109/TITS.2012.2209421>

- [62] Sérgio Moro, Paulo Cortez, and Paulo Rita. 2014. A data-driven approach to predict the success of bank telemarketing. *Decision Support Systems* 62 (2014), 22–31.
- [63] Erica Mourão, João Felipe Pimentel, Leonardo Murta, Marcos Kalinowski, Emilia Mendes, and Claes Wohlin. 2020. On the performance of hybrid search strategies for systematic literature reviews in software engineering. *Information and software technology* 123 (2020), 106294.
- [64] Elizamary Nascimento, Anh Nguyen-Duc, Ingrid Sundbø, and Tayana Conte. 2020. Software engineering for artificial intelligence and machine learning software: A systematic literature review. *arXiv preprint arXiv:2011.03751* (2020).
- [65] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y. Ng. 2011. Reading Digits in Natural Images with Unsupervised Feature Learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning 2011*. http://ufldl.stanford.edu/housenumbers/nips2011_housenumbers.pdf
- [66] Maria-Elena Nilsback and Andrew Zisserman. 2008. Automated flower classification over a large number of classes. In *2008 Sixth Indian Conference on Computer Vision, Graphics & Image Processing*. IEEE, 722–729.
- [67] Parmy Olson. 2011. The algorithm that beats your bank manager. *CNN Money* March 15 (2011).
- [68] Abbas Ourmazd. 2020. Science in the age of machine learning. *Nature Reviews Physics* 2, 7 (2020), 342–343.
- [69] Daniele Ravi, Charence Wong, Fani Deligianni, Melissa Berthelot, Javier Andreu-Perez, Benny Lo, and Guang-Zhong Yang. 2016. Deep learning for health informatics. *IEEE journal of biomedical and health informatics* 21, 1 (2016), 4–21.
- [70] Jörg Rech and Klaus-Dieter Althoff. 2004. Artificial intelligence and software engineering: Status and future trends. *KI* 18, 3 (2004), 5–11.
- [71] Harvard Business Review. 2023. AI Is Not Just Getting Better, it's Becoming More Pervasive. <https://hbr.org/sponsored/2019/02/ai-is-not-just-getting-better-its-becoming-more-pervasive>. Accessed: 2023-03-05.
- [72] Vincenzo Riccio, Gunel Jahangirova, Andrea Stocco, Nargiz Humbatova, Michael Weiss, and Paolo Tonella. 2020. Testing machine learning based systems: a systematic mapping. *Empirical Software Engineering* 25 (2020), 5193–5254.
- [73] Anthony Rousseau, Paul Deléglise, Yannick Esteve, et al. 2014. Enhancing the TED-LIUM corpus with selected data for language modeling and more TED talks.. In *LREC*. 3935–3939.
- [74] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. 2015. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)* 115, 3 (2015), 211–252. <https://doi.org/10.1007/s11263-015-0816-y>
- [75] Alex Serban, Koen van der Blom, Holger Hoos, and Joost Visser. 2020. Adoption and effects of software engineering best practices in machine learning. In *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 1–12.
- [76] Alex Serban and Joost Visser. 2021. An empirical study of software architecture for machine learning. *arXiv preprint arXiv:2105.12422* 39 (2021).
- [77] Yap Yan Siang, Mohd Ridzuan Ahamd, and Mastura Shafinaz Zainal Abidin. 2021. Anomaly detection based on tiny machine learning: A review. *Open International Journal of Informatics* 9, Special Issue 2 (2021), 67–78.
- [78] Miltiadis Siavvas, Dimitrios Tsoukalas, Marija Jankovic, Dionysios Kehagias, and Dimitrios Tzovaras. 2022. Technical debt as an indicator of software security risk: a machine learning approach for software development enterprises. *Enterprise Information Systems* 16, 5 (2022), 1824017.
- [79] Khurram Soomro, Amir Roshan Zamir, and Mubarak Shah. 2012. UCF101: A dataset of 101 human actions classes from videos in the wild. *arXiv preprint arXiv:1212.0402* (2012).
- [80] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel. 2012. Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks* 32 (2012), 323–332. <https://doi.org/10.1016/j.neunet.2012.02.016> Selected Papers from IJCNN 2011.
- [81] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. 2013. Intriguing properties of neural networks. <https://doi.org/10.48550/ARXIV.1312.6199>
- [82] udacity challenge. 2016. Using Deep Learning to Predict Steering Angles. <https://github.com/udacity/self-driving-car>
- [83] Andrew Varga and Herman J.M. Steeneken. 1993. Assessment for automatic speech recognition: II. NOISEX-92: A database and an experiment to study the effect of additive noise on speech recognition systems. *Speech Communication* 12, 3 (1993), 247–251. [https://doi.org/10.1016/0167-6393\(93\)90095-3](https://doi.org/10.1016/0167-6393(93)90095-3)
- [84] David Wagner and Paolo Soto. 2002. Mimicry Attacks on Host-Based Intrusion Detection Systems. In *Proceedings of the 9th ACM Conference on Computer and Communications Security (Washington, DC, USA) (CCS '02)*. Association for Computing Machinery, New York, NY, USA, 255–264. <https://doi.org/10.1145/586110.586145>
- [85] Simin Wang, Liguang Huang, Jidong Ge, Tengfei Zhang, Haitao Feng, Ming Li, He Zhang, and Vincent Ng. 2020. Synergy between machine/deep learning and software engineering: How far are we? *arXiv preprint arXiv:2008.05515* (2020).
- [86] Xianmin Wang, Jing Li, Xiaohui Kuang, Yu-an Tan, and Jin Li. 2019. The security of machine learning in an adversarial setting: A survey. *J. Parallel and Distrib. Comput.* 130 (2019), 12–23.

- [87] Hironori Washizaki, Hiromu Uchida, Foutse Khomh, and Yann-Gaël Guéhéneuc. 2019. Studying software engineering patterns for designing machine learning systems. In *2019 10th International Workshop on Empirical Software Engineering in Practice (IWESEP)*. IEEE, 49–495.
- [88] Claes Wohlin. 2014. Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering (London, England, United Kingdom) (EASE '14)*. Association for Computing Machinery, New York, NY, USA, Article 38, 10 pages. <https://doi.org/10.1145/2601248.2601268>
- [89] Claes Wohlin. 2016. Second-generation systematic literature studies using snowballing. In *Proceedings of the 20th International Conference on Evaluation and Assessment in Software Engineering*. 1–6.
- [90] Claes Wohlin, Emilia Mendes, Katia Romero Felizardo, and Marcos Kalinowski. 2020. Guidelines for the search strategy to update systematic literature reviews in software engineering. *Information and software technology* 127 (2020), 106366.
- [91] Lior Wolf, Tal Hassner, and Itay Maoz. 2011. Face recognition in unconstrained videos with matched background similarity. In *CVPR 2011*. 529–534. <https://doi.org/10.1109/CVPR.2011.5995566>
- [92] Erroll Wood, Tadas Baltrušaitis, Louis-Philippe Morency, Peter Robinson, and Andreas Bulling. 2016. Learning an Appearance-Based Gaze Estimator from One Million Synthesised Images. In *Proceedings of the Ninth Biennial ACM Symposium on Eye Tracking Research & Applications (Charleston, South Carolina) (ETRA '16)*. Association for Computing Machinery, New York, NY, USA, 131–138. <https://doi.org/10.1145/2857491.2857492>
- [93] Zhenqin Wu, Bharath Ramsundar, Evan N Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S Pappu, Karl Leswing, and Vijay Pande. 2018. MoleculeNet: a benchmark for molecular machine learning. *Chemical science* 9, 2 (2018), 513–530.
- [94] Han Xiao, Kashif Rasul, and Roland Vollgraf. 2017. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747* (2017).
- [95] Mengwei Xu, Jiawei Liu, Yuanqiang Liu, Felix Xiaozhu Lin, Yunxin Liu, and Xuanzhe Liu. 2019. A First Look at Deep Learning Apps on Smartphones. In *The World Wide Web Conference (San Francisco, CA, USA) (WWW '19)*. Association for Computing Machinery, New York, NY, USA, 2125–2136. <https://doi.org/10.1145/3308558.3313591>
- [96] Jie M Zhang and Mark Harman. 2021. "Ignorance and Prejudice" in Software Fairness. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1436–1447.
- [97] Jie M Zhang, Mark Harman, Lei Ma, and Yang Liu. 2020. Machine learning testing: Survey, landscapes and horizons. *IEEE Transactions on Software Engineering* 48, 1 (2020), 1–36.
- [98] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. 2019. Deep learning based recommender system: A survey and new perspectives. *ACM computing surveys (CSUR)* 52, 1 (2019), 1–38.
- [99] Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. Character-level convolutional networks for text classification. *Advances in neural information processing systems* 28 (2015).
- [100] Jianlong Zhou and Fang Chen. 2018. *Human and Machine Learning*. Springer.

SLR – SISTEMATIC LITERATURE REVIEW

- [S1] Raluca Maria Hampau, Maurits Kaptein, Robin van Emden, Thomas Rost, and Ivano Malavolta. An empirical study on the performance and energy consumption of ai containerization strategies for computer-vision tasks on the edge. In *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering 2022, EASE '22*, page 50–59, New York, NY, USA, 2022. Association for Computing Machinery.
- [S2] Muhammad Usman, Divya Gopinath, Youcheng Sun, Yannic Noller, and Corina S Păsăreanu. Nn repair: constraint-based repair of neural network classifiers. In *Computer Aided Verification: 33rd International Conference, CAV 2021, Virtual Event, July 20–23, 2021, Proceedings, Part I* 33, pages 3–25. Springer, 2021.
- [S3] Hazem Fahmy, Fabrizio Pastore, Mojtaba Bagherzadeh, and Lionel Briand. Supporting deep neural network safety analysis and retraining through heatmap-based unsupervised learning. *IEEE Transactions on Reliability*, 70(4):1641–1657, 2021.
- [S4] Bing Yu, Hua Qi, Qing Guo, Felix Juefei-Xu, Xiaofei Xie, Lei Ma, and Jianjun Zhao. Deeprepair: Style-guided repairing for deep neural networks in the real-world operational environment. *IEEE Transactions on Reliability*, 71(4):1401–1416, 2022.
- [S5] Hao Zhang and W.K. Chan. Plum: Exploration and prioritization of model repair strategies for fixing deep learning models. In *2021 8th International Conference on Dependable Systems and Their Applications (DSA)*, pages 140–151, 2021.
- [S6] Kevin Eykholt, Ivan Evtimov, Earlene Fernandes, Bo Li, Amir Rahmati, Chaowei Xiao, Atul Prakash, Tadayoshi Kohno, and Dawn Song. Robust physical-world attacks on deep learning visual classification. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.
- [S7] Kexin Pei, Yinzhi Cao, Junfeng Yang, and Suman Jana. Deepxplore: Automated whitebox testing of deep learning systems. In *Proceedings of the 26th Symposium on Operating Systems Principles, SOSP '17*, page 1–18, New York, NY,

- USA, 2017. Association for Computing Machinery.
- [S8] Reza Shokri and Vitaly Shmatikov. Privacy-preserving deep learning. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, CCS '15, page 1310–1321, New York, NY, USA, 2015. Association for Computing Machinery.
 - [S9] Yue Zhao, Hong Zhu, Kai Chen, and Shengzhi Zhang. Ai-lancet: Locating error-inducing neurons to optimize neural networks. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, CCS '21, page 141–158, New York, NY, USA, 2021. Association for Computing Machinery.
 - [S10] Tejas S. Borkar and Lina J. Karam. Deepcorrect: Correcting dnn models against image distortions. *IEEE Transactions on Image Processing*, 28(12):6022–6034, 2019.
 - [S11] Hao Zhang and W.K. Chan. Apricot: A weight-adaptation approach to fixing deep learning models. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 376–387, 2019.
 - [S12] Yingyi Zhang, Zan Wang, Jiajun Jiang, Hanmo You, and Junjie Chen. Toward improving the robustness of deep learning models via model transformation. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, ASE '22, New York, NY, USA, 2023. Association for Computing Machinery.
 - [S13] Zhenpeng Chen, Jie M. Zhang, Federica Sarro, and Mark Harman. Maat: A novel ensemble approach to addressing fairness and performance bugs for machine learning software. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2022, page 1122–1134, New York, NY, USA, 2022. Association for Computing Machinery.
 - [S14] Xiaoning Du, Xiaofei Xie, Yi Li, Lei Ma, Yang Liu, and Jianjun Zhao. Deepstellar: Model-based quantitative analysis of stateful deep learning systems. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2019, page 477–487, New York, NY, USA, 2019. Association for Computing Machinery.
 - [S15] Mengdi Zhang and Jun Sun. Adaptive fairness improvement based on causality analysis. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2022, page 6–17, New York, NY, USA, 2022. Association for Computing Machinery.
 - [S16] Boyuan Chen, Mingzhi Wen, Yong Shi, Dayi Lin, Gopi Krishnan Rajbahadur, and Zhen Ming (Jack) Jiang. Towards training reproducible deep learning models. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, page 2202–2214, New York, NY, USA, 2022. Association for Computing Machinery.
 - [S17] Xuanqi Gao, Juan Zhai, Shiqing Ma, Chao Shen, Yufei Chen, and Qian Wang. Fairneuron: Improving deep neural network fairness with adversary games on selective neurons. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, page 921–933, New York, NY, USA, 2022. Association for Computing Machinery.
 - [S18] Stefanos Georgiou, Maria Kechagia, Tushar Sharma, Federica Sarro, and Ying Zou. Green ai: Do deep learning frameworks have different costs? In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, page 1082–1094, New York, NY, USA, 2022. Association for Computing Machinery.
 - [S19] Yuanchun Li, Jiayi Hua, Haoyu Wang, Chunyang Chen, and Yunxin Liu. Deeppayload: Black-box backdoor attack on deep learning models through neural payload injection. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 263–274, 2021.
 - [S20] Zixi Liu, Yang Feng, Yining Yin, and Zhenyu Chen. Deepstate: Selecting test suites to enhance the robustness of recurrent neural networks. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, page 598–609, New York, NY, USA, 2022. Association for Computing Machinery.
 - [S21] Saeid Tizpaz-Niari, Ashish Kumar, Gang Tan, and Ashutosh Trivedi. Fairness-aware configuration of machine learning libraries. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, page 909–920, New York, NY, USA, 2022. Association for Computing Machinery.
 - [S22] Jingyi Wang, Guoliang Dong, Jun Sun, Xinyu Wang, and Peixin Zhang. Adversarial sample detection for deep neural network through model mutation testing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 1245–1256, 2019.
 - [S23] Jingyi Wang, Jialuo Chen, Youcheng Sun, Xingjun Ma, Dongxia Wang, Jun Sun, and Peng Cheng. Robot: Robustness-oriented testing for deep learning systems. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 300–311, 2021.
 - [S24] Xiyue Zhang, Xiaofei Xie, Lei Ma, Xiaoning Du, Qiang Hu, Yang Liu, Jianjun Zhao, and Meng Sun. Towards characterizing adversarial defects of deep learning software from the lens of uncertainty. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE '20, page 739–751, New York, NY, USA, 2020. Association for Computing Machinery.
 - [S25] Rangeet Pan and Hridesh Rajan. Decomposing convolutional neural networks into reusable and replaceable modules. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, page 524–535, New York, NY, USA, 2022. Association for Computing Machinery.

- [S26] Jie M. Zhang and Mark Harman. "ignorance and prejudice" in software fairness. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 1436–1447, 2021.
- [S27] Pei Huang, Yuting Yang, Minghao Liu, Fuqi Jia, Feifei Ma, and Jian Zhang. Weakened robustness of deep neural networks. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2022*, page 126–138, New York, NY, USA, 2022. Association for Computing Machinery.
- [S28] Pin Ji, Yang Feng, Jia Liu, Zhihong Zhao, and Zhenyu Chen. Asrtest: Automated testing for deep-neural-network-driven speech recognition systems. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2022*, page 189–201, New York, NY, USA, 2022. Association for Computing Machinery.
- [S29] Yu Li, Muxi Chen, and Qiang Xu. Hybridrepair: Towards annotation-efficient repair for deep learning models. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2022*, page 227–238, New York, NY, USA, 2022. Association for Computing Machinery.
- [S30] Jialai Wang, Han Qiu, Yi Rong, Hengkai Ye, Qi Li, Zongpeng Li, and Chao Zhang. Bet: Black-box efficient testing for convolutional neural networks. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2022*, page 164–175, New York, NY, USA, 2022. Association for Computing Machinery.
- [S31] Quan Zhang, Yifeng Ding, Yongqiang Tian, Jianmin Guo, Min Yuan, and Yu Jiang. Advdoor: Adversarial backdoor attack of deep learning system. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2021*, page 127–138, New York, NY, USA, 2021. Association for Computing Machinery.
- [S32] Laurent Gomez, Marcus Wilhelm, José Márquez, and Patrick Duverger. Security for distributed deep neural networks: Towards data confidentiality & intellectual property protection. In *Proceedings of the 16th International Joint Conference on e-Business and Telecommunications - SECRIPT*, pages 439–447. INSTICC, SciTePress, 2019.
- [S33] Hong-Linh Truong and Tri-Minh Nguyen. Qoa4ml - a framework for supporting contracts in machine learning services. In *2021 IEEE International Conference on Web Services (ICWS)*, pages 465–475, 2021.
- [S34] Joymallya Chakraborty, Suvodeep Majumder, Zhe Yu, and Tim Menzies. Fairway: A way to build fair ml software. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2020*, page 654–665, New York, NY, USA, 2020. Association for Computing Machinery.
- [S35] Max Hort, Jie M. Zhang, Federica Sarro, and Mark Harman. Fairea: A model behaviour mutation approach to benchmarking bias mitigation methods. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2021*, page 994–1006, New York, NY, USA, 2021. Association for Computing Machinery.
- [S36] Amin Aminifar, Matin Shokri, Fazle Rabbi, Violet Ka I. Pun, and Yngve Lamo. Extremely randomized trees with privacy preservation for distributed structured health data. *IEEE Access*, 10:6010–6027, 2022.
- [S37] Lizhi Liao, Heng Li, Weiyi Shang, and Lei Ma. An empirical study of the impact of hyperparameter tuning and model optimization on the performance properties of deep neural networks. *ACM Trans. Softw. Eng. Methodol.*, 31(3), apr 2022.
- [S38] Deqing Zou, Yawei Zhu, Shouhuai Xu, Zhen Li, Hai Jin, and Hengkai Ye. Interpreting deep learning-based vulnerability detector predictions based on heuristic searching. *ACM Trans. Softw. Eng. Methodol.*, 30(2), mar 2021.
- [S39] Wenbo Guo, Dongliang Mu, Jun Xu, Purui Su, Gang Wang, and Xinyu Xing. Lemna: Explaining deep learning based security applications. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS '18*, page 364–379, New York, NY, USA, 2018. Association for Computing Machinery.
- [S40] Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 39–57, 2017.
- [S41] Timon Gehr, Matthew Mirman, Dana Drachler-Cohen, Petar Tsankov, Swarat Chaudhuri, and Martin Vechev. Ai2: Safety and robustness certification of neural networks with abstract interpretation. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 3–18, 2018.
- [S42] Xiaowei Huang, Marta Kwiatkowska, Sen Wang, and Min Wu. Safety verification of deep neural networks. In *Computer Aided Verification: 29th International Conference, CAV 2017, Heidelberg, Germany, July 24–28, 2017, Proceedings, Part I 30*, pages 3–29. Springer, 2017.
- [S43] Yujin Huang, Han Hu, and Chunyang Chen. Robustness of on-device models: Adversarial attack to deep learning models on android apps. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 101–110, 2021.
- [S44] Wei Huang, Youcheng Sun, Xingyu Zhao, James Sharp, Wenjie Ruan, Jie Meng, and Xiaowei Huang. Coverage-guided testing for recurrent neural networks. *IEEE Transactions on Reliability*, 71(3):1191–1206, 2022.
- [S45] Yujin Huang and Chunyang Chen. Smart app attack: Hacking deep learning models in android apps. *IEEE Transactions on Information Forensics and Security*, 17:1827–1840, 2022.
- [S46] Aniya Aggarwal, Pranay Lohia, Seema Nagar, Kuntal Dey, and Diptikalyan Saha. Black box fairness testing of machine learning models. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering*

Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2019, page 625–635, New York, NY, USA, 2019. Association for Computing Machinery.

- [S47] Maxime Cordy, Steve Muller, Mike Papadakis, and Yves Le Traon. Search-based test and improvement of machine-learning-based anomaly detection systems. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2019, page 158–168, New York, NY, USA, 2019. Association for Computing Machinery.
- [S48] Sumon Biswas and Hridesh Rajan. Fair preprocessing: Towards understanding compositional fairness of data transformers in machine learning pipeline. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2021, page 981–993, New York, NY, USA, 2021. Association for Computing Machinery.
- [S49] Florian Tramèr, Vaggelis Atlidakis, Roxana Geambasu, Daniel Hsu, Jean-Pierre Hubaux, Mathias Humbert, Ari Juels, and Huang Lin. Fairtest: Discovering unwarranted associations in data-driven applications. In *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 401–416, 2017.
- [S50] Joy Buolamwini and Timnit Gebru. Gender shades: Intersectional accuracy disparities in commercial gender classification. In Sorelle A. Friedler and Christo Wilson, editors, *Proceedings of the 1st Conference on Fairness, Accountability and Transparency*, volume 81 of *Proceedings of Machine Learning Research*, pages 77–91. PMLR, 23–24 Feb 2018.